

第3讲：常用的中规模组合逻辑电路

1 译码器

功能分类：

- 变量译码器：用来表示输入变量状态的全部组合，N位输入， 2^N 输出。常见的集成化译码器有2-4、3-8、4-16
实现的功能就是：二进制地址->地址能表示的种数（这些地址能选的全部种数）
- 码制译码器：如8421码变换为循环码等
- 显示译码器：控制数码管显示

1.1 2-4变量译码器

1.1.1 2-4译码器

- 定义：2 - 4译码器是指2输入-4输出的变量译码器。2输入，4输出。对应输入的每一种组合，唯一只有一个输出为“1”（选上的）
- 1 画出真值表 ---> 2 根据真值表写出逻辑表达式（复杂时需要借助卡诺图）
---> 3 画出逻辑电路图

真值表

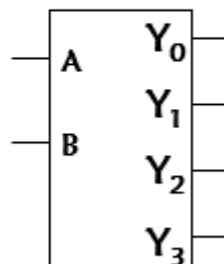
输 入		输 出			
A	B	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
1	0	0	1	0	0
0	1	0	0	1	0
1	1	0	0	0	1

输出表达式

$$\begin{cases} Y_0 = \overline{A}\overline{B} \\ Y_1 = \overline{A}B \\ Y_2 = A\overline{B} \\ Y_3 = AB \end{cases}$$

只用与非门实现

逻辑示意图

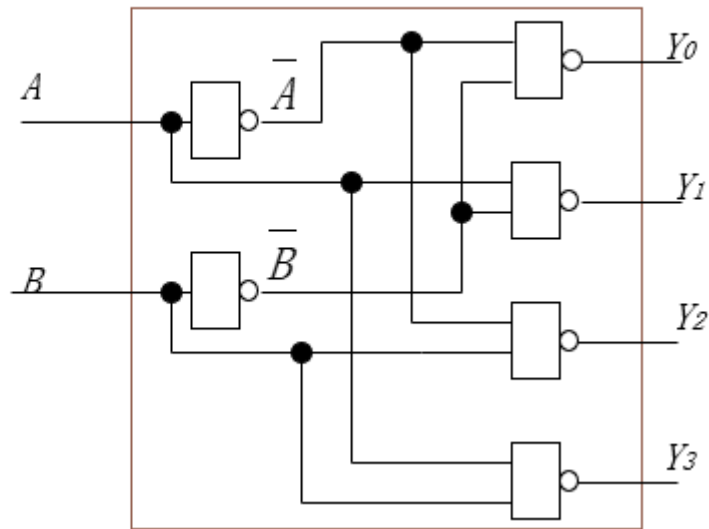


□ (步骤三) 按照输出表达式画出逻辑图

输出表达式

$$\begin{cases} Y_0 = \overline{\overline{AB}} \\ Y_1 = \overline{AB} \\ Y_2 = \overline{AB} \\ Y_3 = \overline{AB} \end{cases}$$

有没有什么问题？

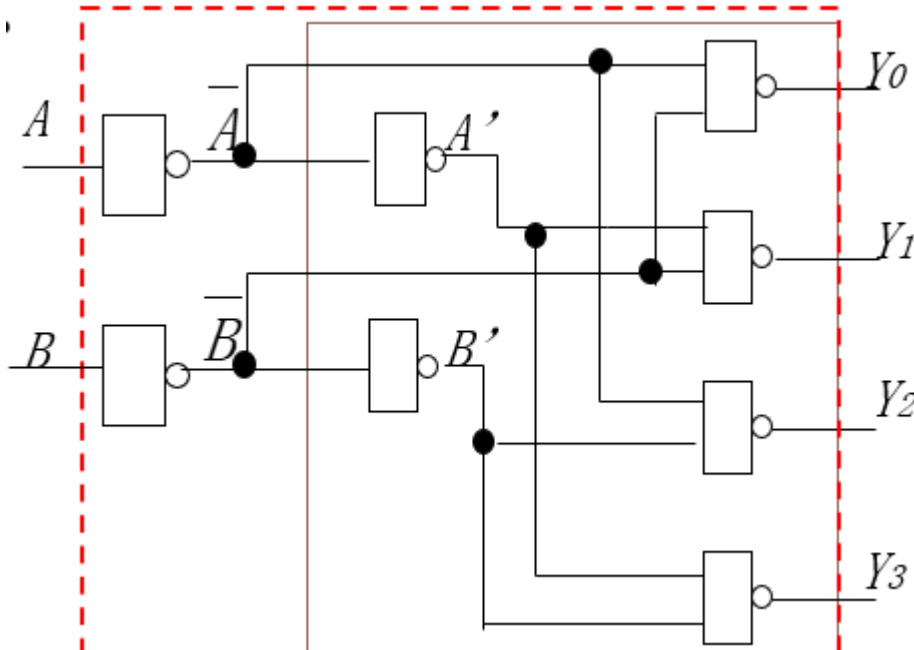


步骤4：检查可能出现的问题：(考试时不专门说明不用考虑)

如上面的电路中，A，B的输入端各有三个负载！

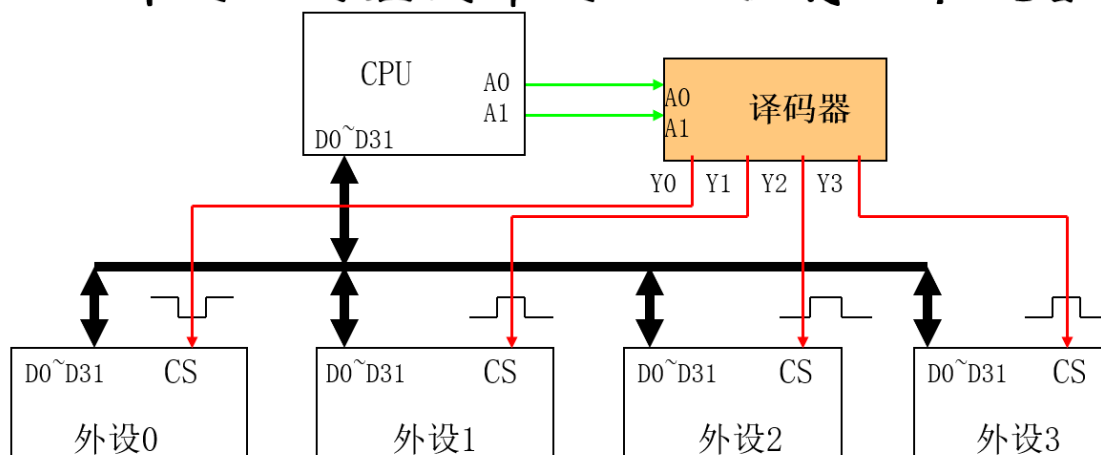
集成电路的设计原则：一个输入只能按照一个输入负载计算！

修改：增加一级输入缓冲，使得对外负载只有一个，减轻上一级电路的负担



3. 2-4译码器应用举例：

■ 2-4译码器的应用举例：CPU控制四个设备



功能
级设
计要
求：

13

A0=0, A1=0时, 外设0工作
A0=1, A1=0时, 外设1工作
A0=0, A1=1时, 外设2工作
A0=1, A1=1时, 外设3工作

信号
级设
计要
求：

A0=0, A1=0时, Y0=0, Y1, Y2, Y3=1
A0=1, A1=0时, Y1=0, Y0, Y2, Y3=1
A0=0, A1=1时, Y2=0, Y0, Y1, Y3=1
A0=1, A1=1时, Y3=0, Y0, Y1, Y2=1

1.1.2 有使能端的2-4译码器

✎ 问题

2 - 4译码器的4个输出是2输入的逻辑组合，任何一种组合都会有一个输出有效。我们希望有一种情况能让所有输出都无效，于是增加附加逻辑——使能

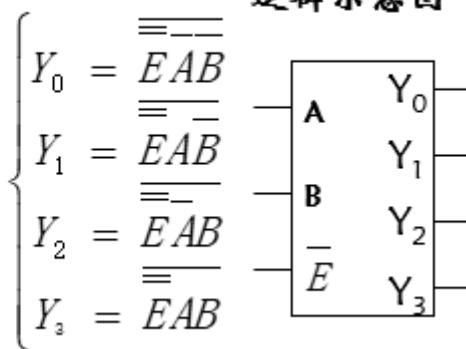
1. 有使能端 $\bar{E}$ 的2-4译码器：

- 当 $\bar{E} = 0$ ，译码器使能。当 $\bar{E} = 1$ ，译码器禁止。
- 画出真值表：

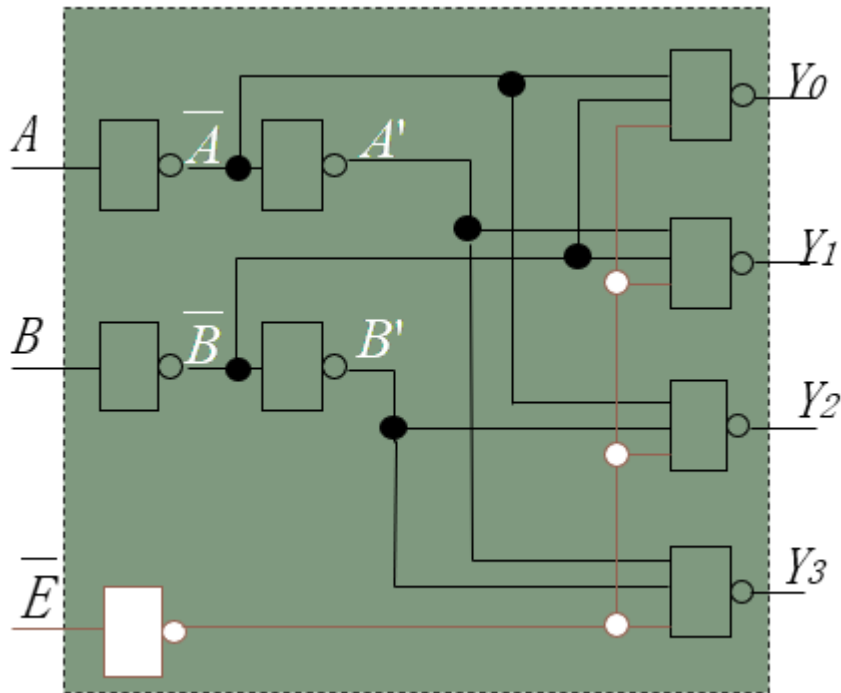
功能表

$\bar{E}$	A	B	Y_0	Y_1	Y_2	Y_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	1	0	1	0	1	1
0	0	1	1	1	0	1
0	1	1	1	1	0	0

逻辑示意图



- 画出逻辑电路图：



1.1.3 使能端的作用

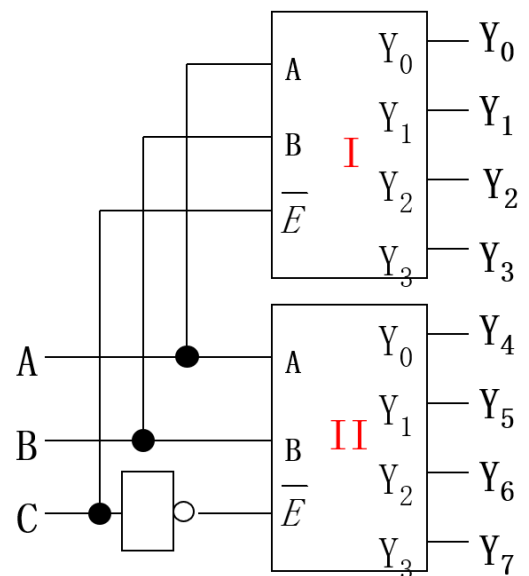
- 用于多片扩展：

三 多片扩展

用两片带使能端的2-4译码器组成3-8译码器。高位输入C用作选片，连接两个2-4译码器的使能端。容易看出：C = 0选中片1，C = 1选中片2

真 值 表

输 入			输 出							
A	B	C	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	0	0	0	1	1	1	1	1	1	1
1	0	0	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
1	1	0	1	1	1	0	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
0	1	1	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0



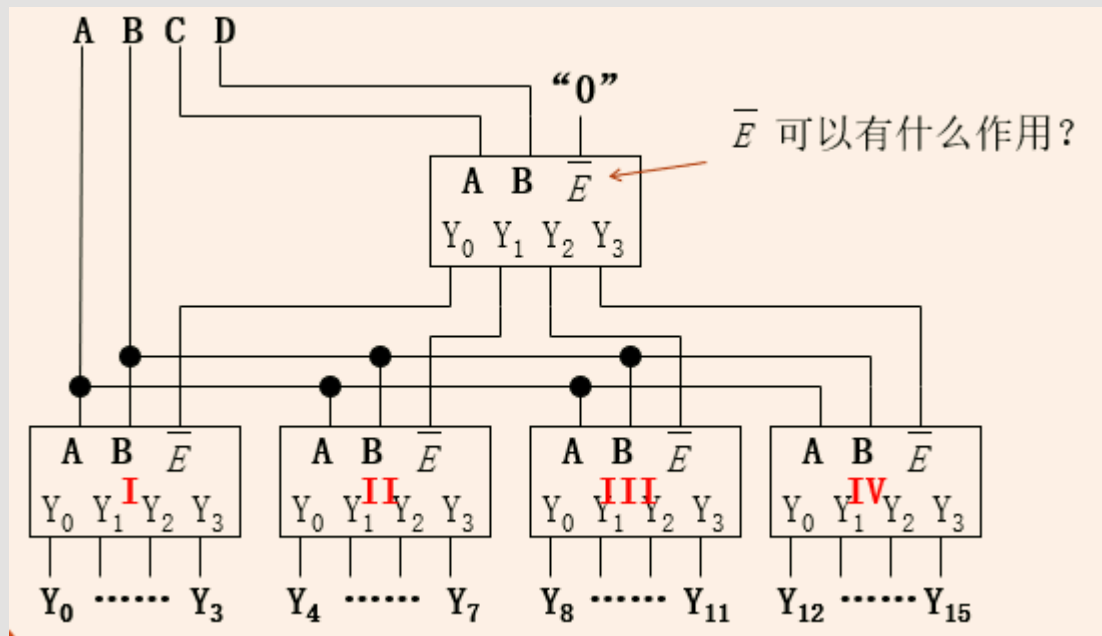
20

思考

用2 - 4译码器构成4 - 16译码器，需要几片2-4译码器？

answer >

5片，如下图：



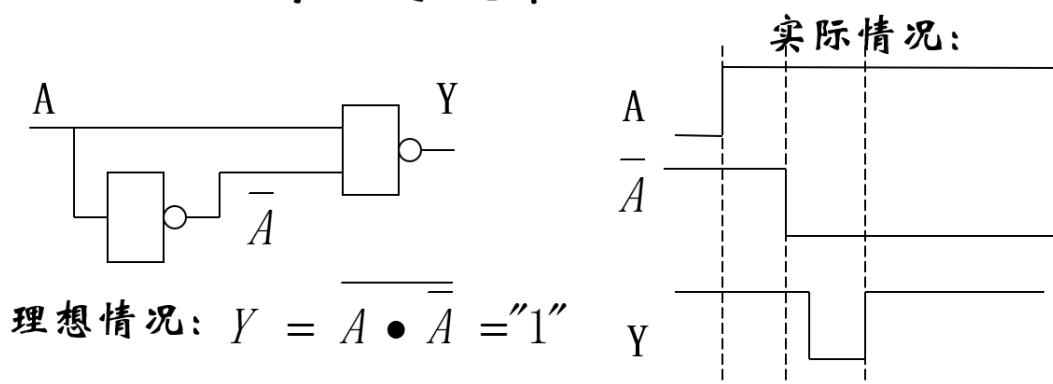
C,D端通过另一个2-4译码器输出操控另外4个2-4译码器的信号，第一个 $\bar{E}$ 可以控制使能全部输出无效。

2. $\bar{E}$ 用作选通

- 针对门电路的传输延迟造成的竞争、冒险问题提出的

三 延迟产生的尖峰信号（毛刺）

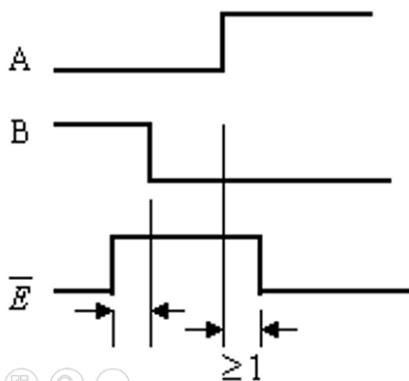
二输入与非门的输入为 A 和 $\bar{A}$ 时， $\bar{A}$ 滞后于 A ，则 Y 会出现尖峰信号（与非门上升沿有尖峰）



如图：图示为负向尖峰。PS：Y门也有延迟

- 2 - 4译码器中设置二级缓冲，目的是均衡负载。但是由于信号传输的延迟，会在输出端产生“0”重叠(Overlap)和尖峰信号。 $\bar{E}$ 的作用是消除尖峰和重叠。

□ 在A B变化期间，输出是不稳定的，可能会出现尖峰信号。加一个能覆盖输入变化的正脉冲 ($\bar{E} = 1$)，使得A B变化期间强制 $Y_0 \sim Y_3 = 1$ ，即可消除输出端的干扰。



抑制尖峰和零重叠的使能正信号应先于（或同时）译码器的变量输入变化前到来，正信号撤除应滞后于变量输入的变化（至少滞后1级缓冲的延迟）。

但也不能太宽，否则速度会慢。

1.2 3-8译码器

- 定义：3输入，8输出
- 真值表和逻辑表达式：

► 真值表和逻辑表达式

真 值 表

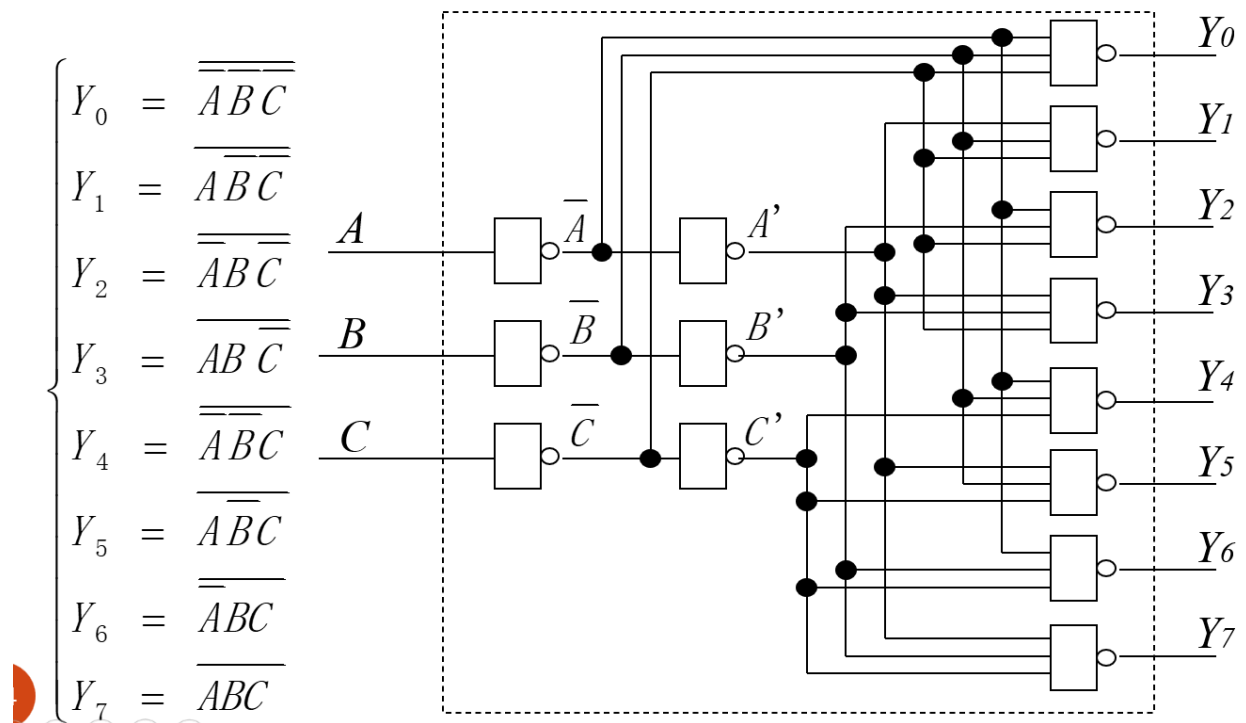
输 入			输 出							
A	B	C	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	0	0	0	1	1	1	1	1	1	1
1	0	0	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
1	1	0	1	1	1	0	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
0	1	1	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0

输出表达式

$$\begin{cases} Y_0 = \overline{A} \overline{B} \overline{C} \\ Y_1 = \overline{A} \overline{B} C \\ Y_2 = \overline{A} B \overline{C} \\ Y_3 = \overline{A} B C \\ Y_4 = A \overline{B} \overline{C} \\ Y_5 = A \overline{B} C \\ Y_6 = A B \overline{C} \\ Y_7 = A B C \end{cases}$$

只
用
非
门
实
现
的
输
出
表
达
式

3. 逻辑门电路：（含缓冲）



4. 3-8译码器也可以扩展成4-16译码器，扩展方法和前面类似，也是借助于使能端，用两个3-8译码器，第四个输入用于选择第一个还是第二个3-8译码器
5. 译码器可做数据分配器

1.3 4-16译码器

⚠ 存在的问题

缓冲门和使能端的负载过大，如图：第一级缓冲门(反变量) 负载9个负载，第二级缓冲门(原变量) 8个负载；使能端与门的负载有16个

事实上，若输入变量数为N，单级译码器的负载：

- 第一级缓冲门为 $2^{N-1} + 1$ ，第二级缓冲门为 2^{N-1} ，使能端为 2^N
- 解决方式：多级译码

多级译码：

多级译码

$$\text{令 } E = \overline{AB}, F = \overline{AB}, G = \overline{AB}, H = AB$$

$$W = \overline{CD}, X = \overline{CD}, Y = \overline{CD}, Z = CD, \text{ 则有:}$$

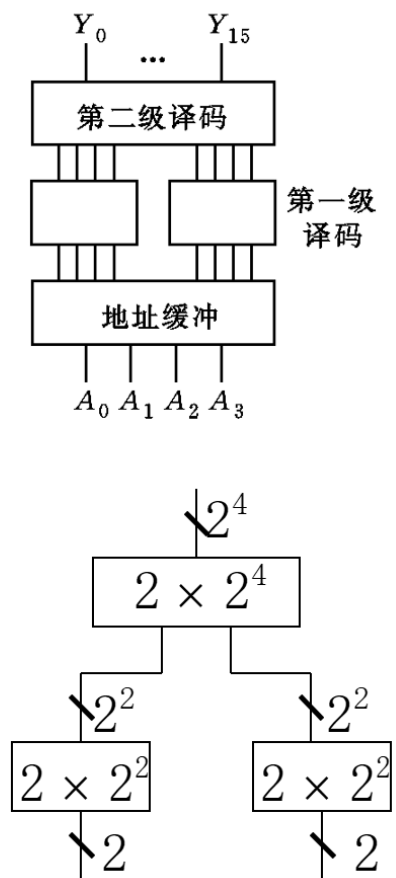
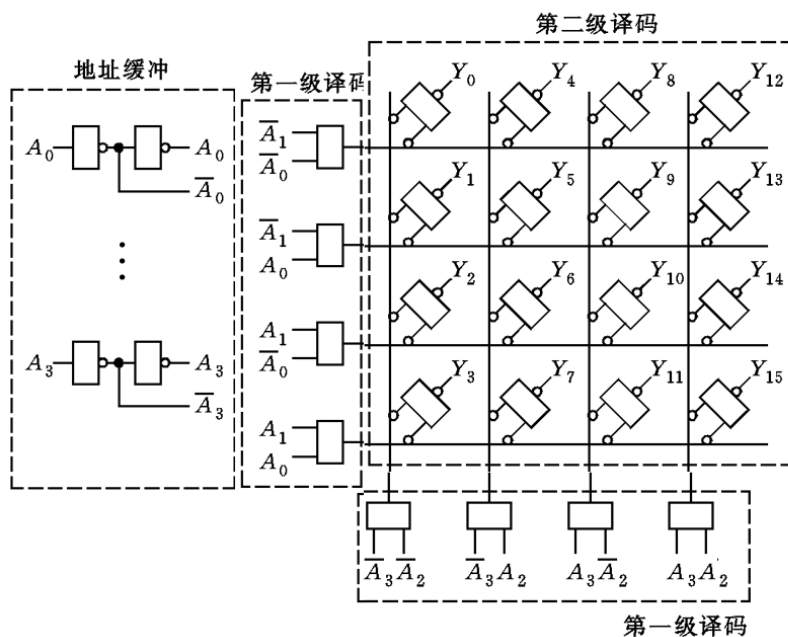
$Y_0 = \overline{\overline{ABCD}} = \overline{EW}$	$Y_4 = \overline{\overline{ABCD}} = \overline{EX}$	$Y_8 = \overline{\overline{ABCD}} = \overline{EY}$	$Y_{12} = \overline{\overline{ABCD}} = \overline{EZ}$
$Y_1 = \overline{\overline{ABCD}} = \overline{FW}$	$Y_5 = \overline{\overline{ABCD}} = \overline{FX}$	$Y_9 = \overline{\overline{ABCD}} = \overline{FY}$	$Y_{13} = \overline{\overline{ABCD}} = \overline{FZ}$
$Y_2 = \overline{\overline{ABCD}} = \overline{GW}$	$Y_6 = \overline{\overline{ABCD}} = \overline{GX}$	$Y_{10} = \overline{\overline{ABCD}} = \overline{GY}$	$Y_{14} = \overline{\overline{ABCD}} = \overline{GZ}$
$Y_3 = \overline{\overline{ABCD}} = \overline{HW}$	$Y_7 = \overline{\overline{ABCD}} = \overline{HX}$	$Y_{11} = \overline{\overline{ABCD}} = \overline{HY}$	$Y_{15} = \overline{\overline{ABCD}} = \overline{HZ}$

✓ A,B,C,D和它们的反变量，都出现8次，说明它们共有8个负载。

✓ 考察E,F,G,H,W,X,Y,Z，每个出现4次，意味着它们共有4个负载。

二级译码

用两级译码电路实现4-16译码器



(2×2^2 表示2输入与门4个, 2×2^4 表示2输入与非门16个)

1.4 码制译码器

1. 定义：将一种编码变换为另外一种编码的逻辑电路

2. 二 - 十进制编码器：二进制编码的十进制数，也叫BCD编码

1.4.1 不完全译码的BCD译码器

8-4-2-1 码表示十进制数

利用8421码表示0~9，而大于9的我们不关心。即ABCD输入表示的数超过0101时， Y_{0-9} 为任意值。这就用到我们之前学的含无关项的卡诺图法[第1章：逻辑代数与化简方法 > 2.3.4 特殊形式的逻辑函数化简](#)

8-4-2-1 码表示十进制数

B \ A	00	01	11	10
D \ C				
00	0	1	3	2
01	4	5	7	6
11	X	X	X	X
10	8	9	X	X

十进制数	8421码
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

如：当ABCD=1001，表示9，即 $Y_9 = 0, Y_{0-8} = 1$ ，在此基础上化简 Y_9 ，利用无关项，容易化简得 $Y_9 = \bar{A}D$ 。

B \ A	00	01	11	10
D \ C				
00	$Y_0=0$	$Y_1=0$	$Y_3=0$	$Y_2=0$
01	$Y_4=0$	$Y_5=0$	$Y_7=0$	$Y_6=0$
11	X	X	X	X
10	$Y_8=0$	$Y_9=0$	X	X

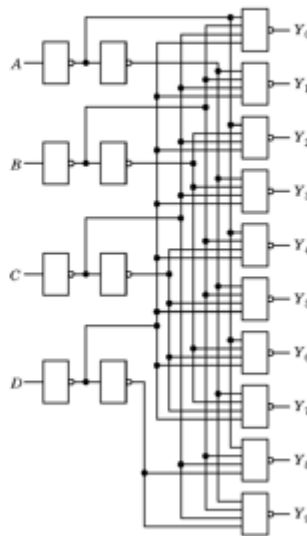
$$\left\{ \begin{array}{l} Y_0 = \overline{\overline{A}\overline{B}\overline{C}\overline{D}} \\ Y_1 = \overline{\overline{A}\overline{B}CD} \\ Y_2 = \overline{\overline{A}BC} \\ Y_8 = \overline{\overline{A}D} \\ Y_9 = \overline{\overline{A}D} \end{array} \right.$$

1.4.2 完全译码的BCD译码器

完全译码，则要求ABCD输入表示的数超过0101时，译码器输出 Y_{0-9} 均为1。

● 完全译码的BCD译码器逻辑图

$$\begin{cases} Y_0 = \overline{A}\overline{B}\overline{C}\overline{D} \\ Y_1 = \overline{A}\overline{B}CD \\ Y_2 = \overline{A}B\overline{C}\overline{D} \\ Y_3 = \overline{A}B\overline{C}D \\ Y_4 = \overline{A}BC\overline{D} \\ Y_5 = \overline{A}BCD \\ Y_6 = A\overline{B}\overline{C}\overline{D} \\ Y_7 = A\overline{B}\overline{C}D \\ Y_8 = A\overline{B}C\overline{D} \\ Y_9 = A\overline{B}CD \end{cases}$$



(b) 逻辑图

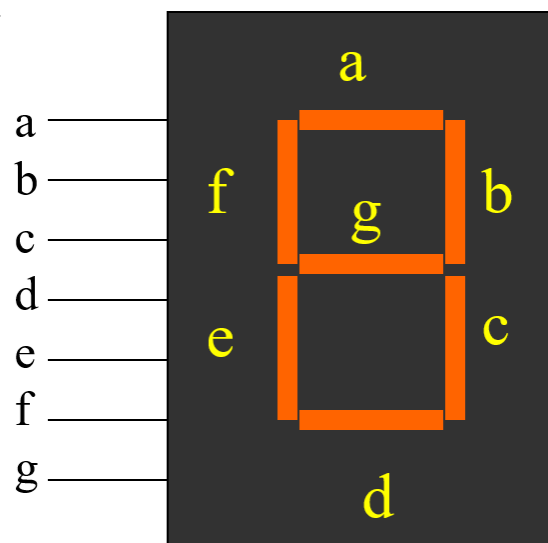
1.5 显示译码器

✎ 问题情境

如图所示的是数码管数字显示器，可以显示0~9不同的数字

■ 显示译码器

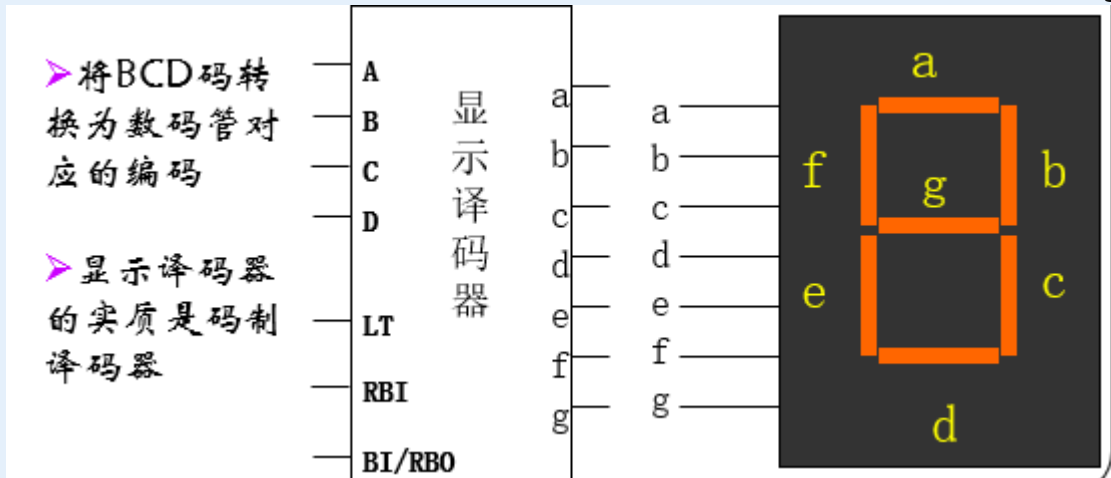
- 一个七段数码管有7个控制端输入a~g，分别对应与数码管的7段。有些数码管是8个输入，在右下方有一个小数点。



数码管的显示类型有两种：

- 共阳极：管脚为“低(0)”，则对应的段点亮，为“高(1)”则灭
- 共阴极：管脚为“高”，则对应的段点亮，为“低”则灭，这里采用共阳极问题是：

8421BCD码的输入ABCD对应着0~9，我们想让它数码管显示器上显示出来，也就是说我们需要一个显示译码器实现：BCD码(ABCD)->显示码(abcdefg)



1. 解决步骤：

因为每个输出 (a,b,...) 在不同的ABCD输入下取值比前面更复杂，不再是“一次0剩下全是1”，而是“有时是0有时是1”，所以得用卡诺图逐个变量化简。如：

● 写a的逻辑表达式

		B			
		A			
D	C	00	01	11	10
	00	0	1	0	0
	01	1	0	0	1
	11	1	0	1	1
	10	0	0	1	1

$$a = BD + \overline{A}C + \overline{A}\overline{B}\overline{C}\overline{D}$$

用这样的方法写出所有变量的逻辑表达式：（可作为练习）

$$a = BD + \overline{AC} + \overline{A}\overline{B}\overline{C}\overline{D}$$

$$b = BD + \overline{A}BC + \overline{A}\overline{B}C$$

$$c = \overline{A}\overline{B}\overline{C} + \overline{C}\overline{D}$$

$$d = \overline{A}BC + \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C}$$

$$e = A + \overline{BC}$$

$$f = \overline{A}\overline{C}\overline{D} + \overline{A}B + \overline{B}\overline{C}$$

$$g = \overline{B}\overline{C}\overline{D} + \overline{A}BC$$

然后画出电路图即可。

2 数据选择器

✎ 概念

- 在选择控制的信号作用下，能从多个输入数据中选择一个或多个作为输出。
- 多输入单输出数据选择器
- 多输入多输出数据选择器

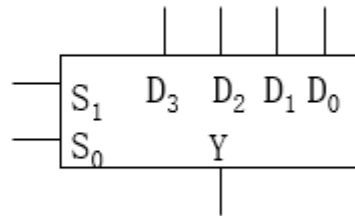
2.1 4选1数据选择器

☰ Example

4通道输入，1输出，2个选择控制

4输入，1输出，2个选择控制

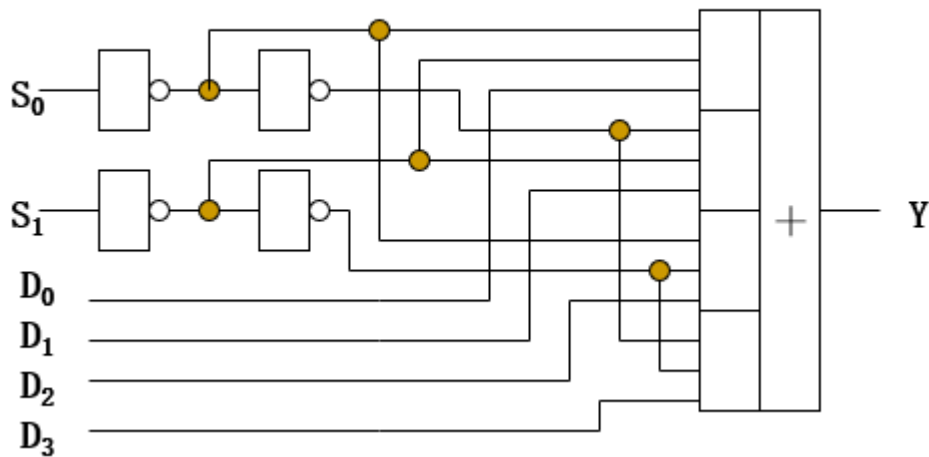
S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3



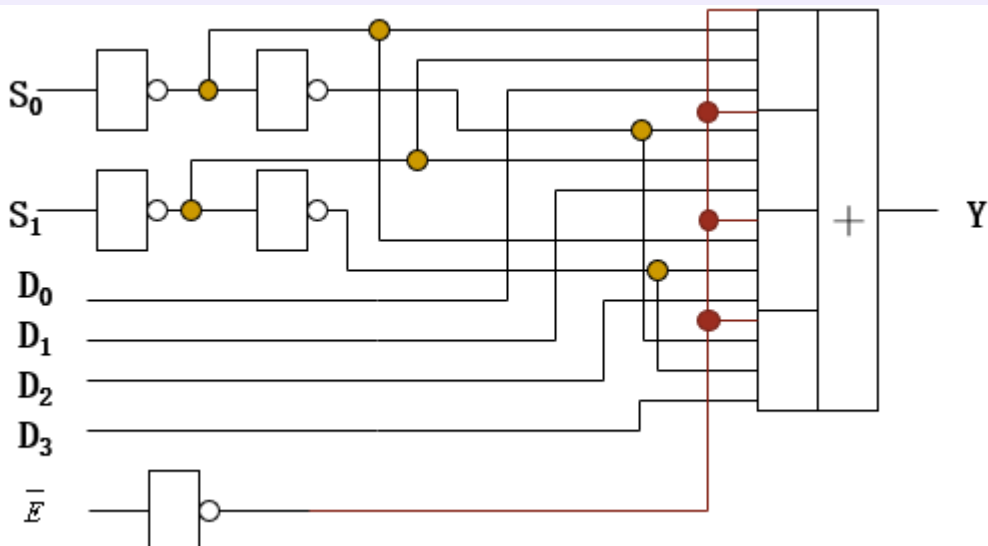
$$Y = \overline{S_0}\overline{S_1}D_0 + S_0\overline{S_1}D_1 + \overline{S_0}S_1D_2 + S_0S_1D_3$$

逻辑电路图：

$$Y = \overline{S_0}\overline{S_1}D_0 + S_0\overline{S_1}D_1 + \overline{S_0}S_1D_2 + S_0S_1D_3$$



带使能端的数据选择器：



注意：使能端都是0允许，1禁止

2.2 数据选择器的扩展

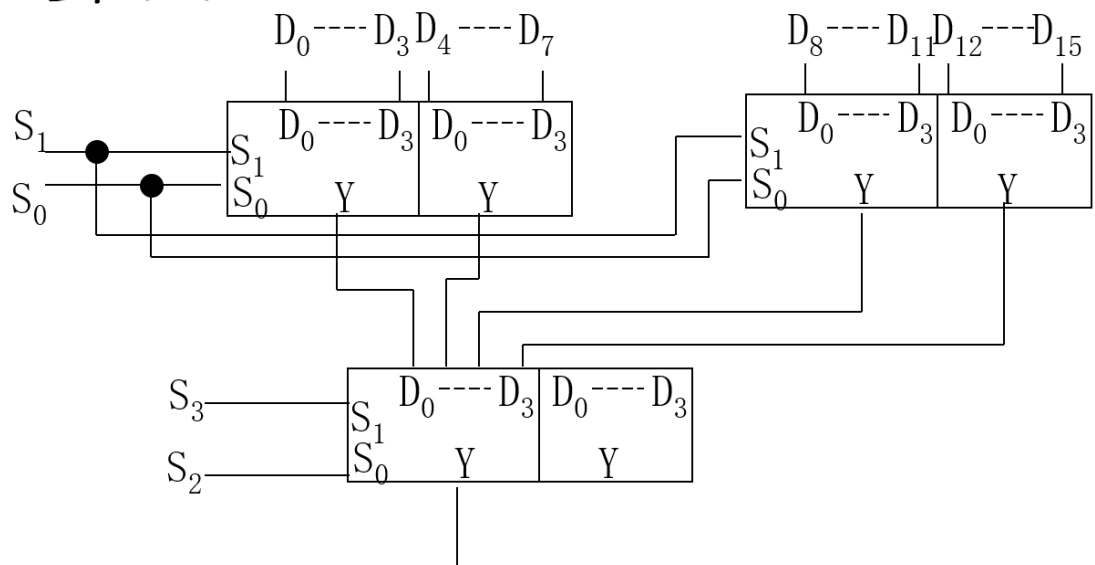
1. 数据选择器在理解时可以和译码器进行类比。
 - 比如，双4选1数据选择器需要用2个选择控制，从4个输入中选择一个输出。这不正向是2-4译码器输入2个，输出的4个中只有1个为1（选择一个）吗？
 - 同理，8选1就需要3个控制端，16选1需要4个

三 用双4选1选择器扩展成16选1选择器

思路：分两层选择：

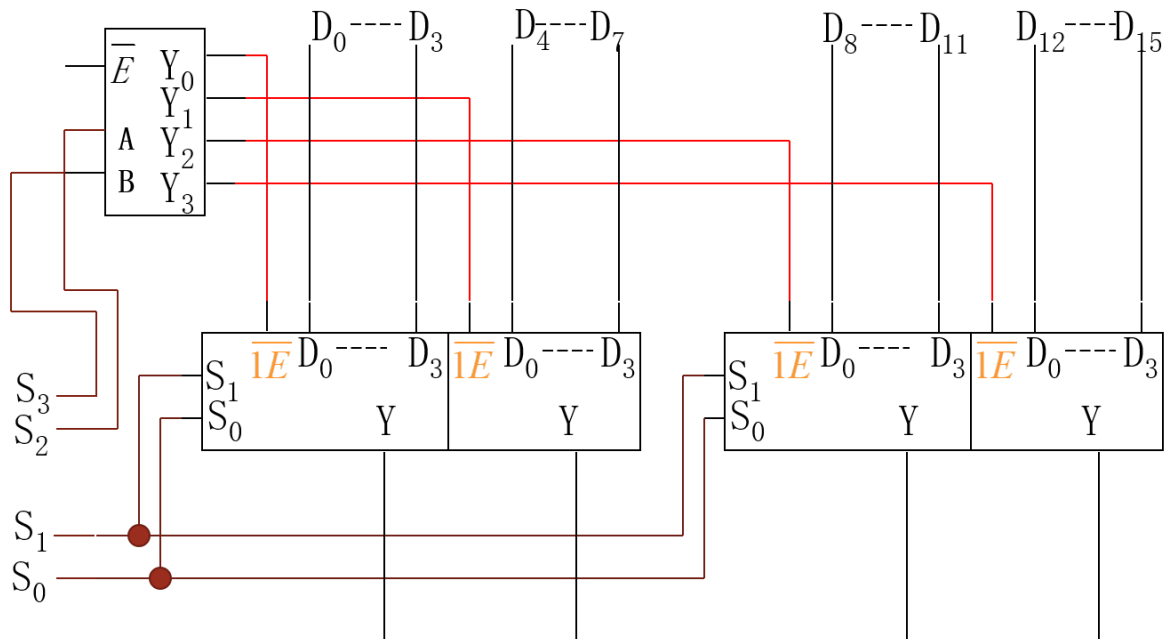
- 第一层选大组（比如选中4个 D_0 ，就是选中 $D_0D_4D_8D_{12}$ 这一组），第二层具体选择

两级选择结构



- 逻辑结构： $S_1 S_0$ 控制第一层选择， $S_3 S_2$ 控制第二层选择。

如果带使能端，则：



高两位控制端经译码后分别控制数据选择器的使能端E，
以实现扩展。此处的输出级是三态门，因此可以“线与”。

2.3 对译码器和数据选择器的进一步理解

1. 利用译码器实现逻辑函数

- 译码器：可以看成是N个输入变量组成的 2^N 个最小项
- 如果再加一级与非门，可组成“与非-与非”逻辑，也可表达“与-或”逻辑。即可用译码器实现“与-或”逻辑函数。（因为两个与非再做与非就是与或）

Example

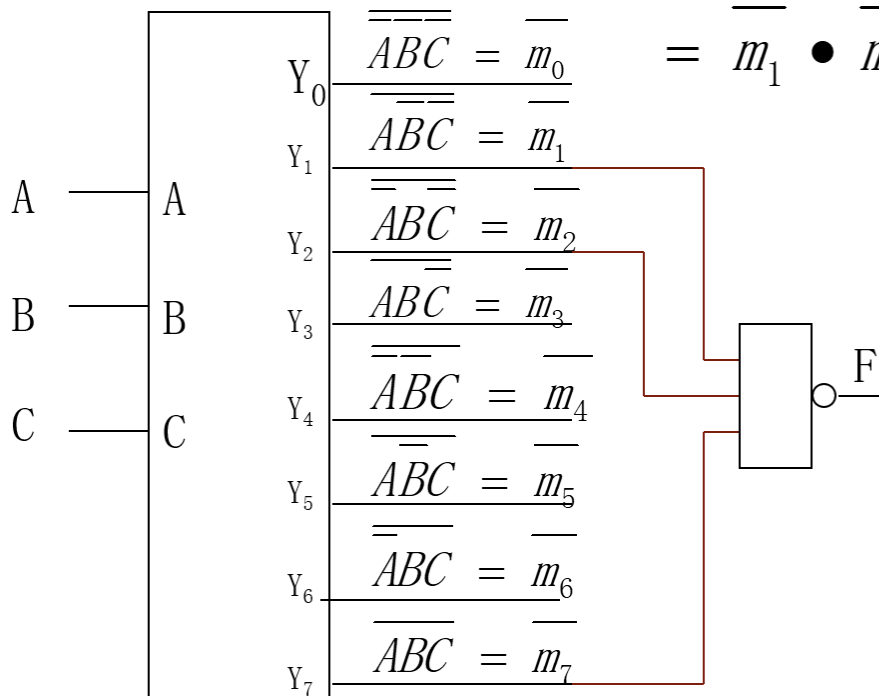
用译码器实现如下函数：

$$F = A\bar{B}\bar{C} + \bar{A}B\bar{C} + ABC$$

三个变量，实际就是用3-8译码器，译码器的8个输出就是三个变量的所有最小项的非，再挑出函数需要的几个最小项做与非，就得到与或了。

$$F = \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} = m_1 + m_2 + m_7$$

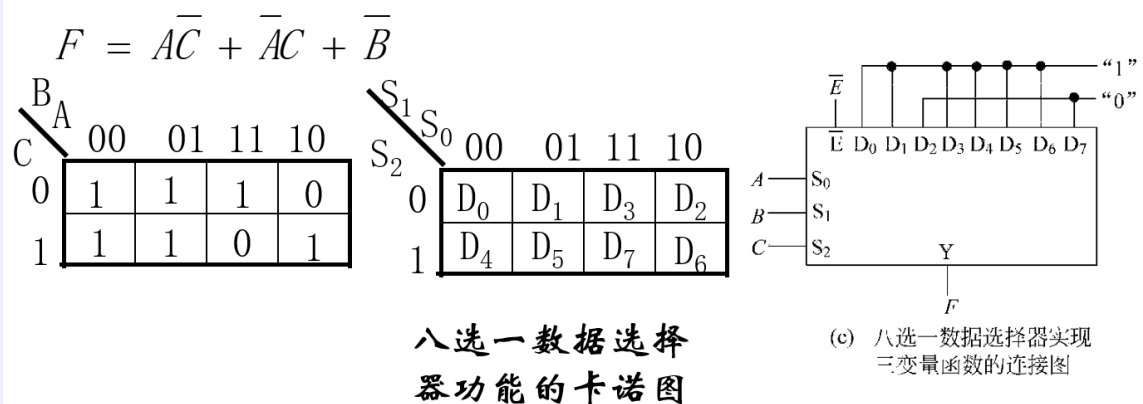
$$= \overline{\overline{\overline{A} \cdot \overline{B} \cdot \overline{C}}} = \overline{\overline{A} \cdot \overline{B} \cdot \overline{C}} = m_1 \cdot m_2 \cdot m_7$$



2. 利用数据选择器实现逻辑函数

- 数据选择器：逻辑结构就是与-或表达式
- 数据选择器可以看成是N个控制端的 2^N 个最小项（下面的 S_0S_1 项）和 2^N 个输入（下面的 D_i 项）组成的“与-或”表达式。令某些输入为“1”，就是选中这些最小项组成逻辑函数。例如四选一： $Y = \bar{S}_1\bar{S}_2D_0 + S_0\bar{S}_1D_1 + \bar{S}_0S_1D_2 + S_0S_1D_3$

Example



八选一数据选择器功能的卡诺图

注意对应关系

思考题

用8选1的数据选择器怎么实现4变量函数？

answer

- 重点在于真正理解前面数据选择器实现逻辑函数的内涵。数据选择器实际上就是实现了一个包含控制变量和输入变量的逻辑函数，它是控制变量的最小项和输入变量的与的或，比如4选1的 $Y = \bar{S}_1\bar{S}_2D_0 + S_0\bar{S}_1D_1 + \bar{S}_0S_1D_2 + S_0S_1D_3$ ，这时候我如果令我需要的几个最小项的 $D_i = 1$ ，其他的D均为0，这个逻辑函数就退化为我要实现的逻辑函数。
- 那么，实际上并不需要这样，我的目的只是让原本数据选择器的逻辑函数退化即可。对于我需要实现的四变量逻辑函数，我可以先选定其中3个（如A,B,C）为控制变量，那么对第四个变量N，含N的与项，我就让对应的 $D_i =$ 相应的N项，不含N的与项，那还是一样的退化方式（对应的 D_i 等于1），这样就实现了四变量函数。

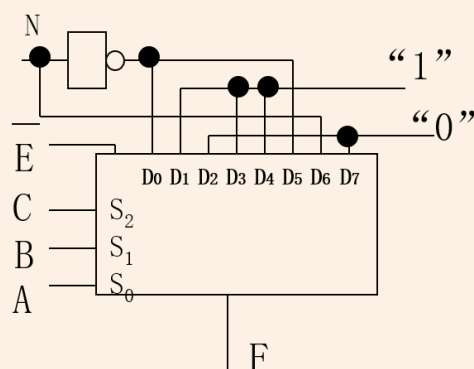
例如：

译码器与数据选择器实现逻辑函数：8选1数据

选择器可以实现4变量函数 $F = \bar{A}\bar{C} + \bar{B}\bar{N} + \bar{A}CN$

令 $A=S_0, B=S_1, C=S_2$

S_0	S_1	S_2	输入	函数F的值
0	0	0	D_0	$F = \bar{A}\bar{C} + \bar{B}\bar{N} + \bar{A}CN = \bar{N}$
1	0	0	D_1	$F = \bar{A}\bar{C} + \bar{B}\bar{N} + \bar{A}CN = 1$
0	1	0	D_2	$F = \bar{A}\bar{C} + \bar{B}\bar{N} + \bar{A}CN = 0$
1	1	0	D_3	$F = \bar{A}\bar{C} + \bar{B}\bar{N} + \bar{A}CN = 1$
0	0	1	D_4	$F = \bar{A}\bar{C} + \bar{B}\bar{N} + \bar{A}CN = 1$
1	0	1	D_5	$F = \bar{A}\bar{C} + \bar{B}\bar{N} + \bar{A}CN = \bar{N}$
0	1	1	D_6	$F = \bar{A}\bar{C} + \bar{B}\bar{N} + \bar{A}CN = N$
1	1	1	D_7	$F = \bar{A}\bar{C} + \bar{B}\bar{N} + \bar{A}CN = 0$



91

3 编码器

编码器(Encoder)

编码器：将译码器反过来，对应输入的每一个状态，输出一个编码

实际信息 $\Rightarrow$ 二进制编码

编码器实际上需要限制输入的状态，某一个时刻只能有一个“1”或者一个“0”

常见的编码器：

- 4-2编码器：将输入的4个状态编成2位二进制数码；
- 8-3编码：

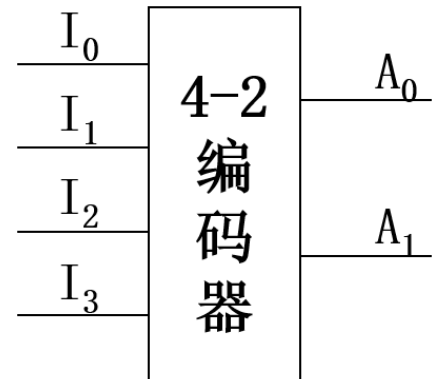
- BCD编码：将10个输入编成BCD码

3.1 4-2 编码器

1. 功能表和逻辑表达式

功能表

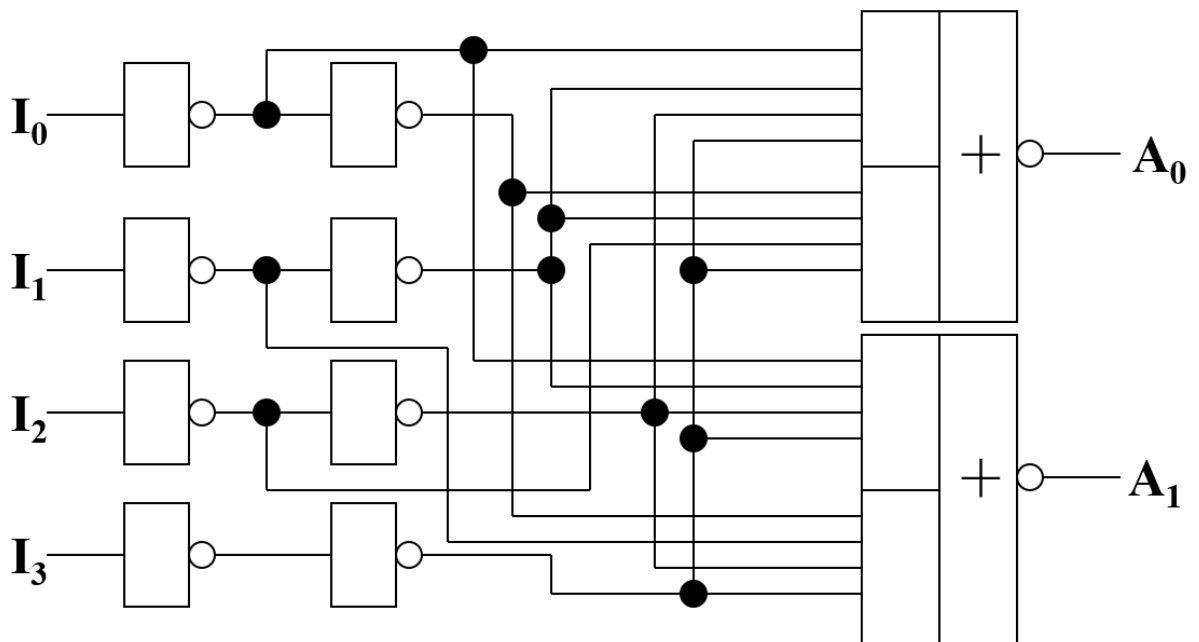
I_0	I_1	I_2	I_3	A_0	A_1
0	1	1	1	0	0
1	0	1	1	1	0
1	1	0	1	0	1
1	1	1	0	1	1



$$\begin{cases} A_0 = I_0 \bar{I}_1 I_2 I_3 + I_0 I_1 \bar{I}_2 \bar{I}_3 = \overline{\bar{I}_0 I_1 I_2 I_3 + I_0 I_1 \bar{I}_2 I_3} \\ A_1 = I_0 I_1 \bar{I}_2 I_3 + I_0 I_1 I_2 \bar{I}_3 = \overline{\bar{I}_0 I_1 I_2 I_3 + I_0 \bar{I}_1 I_2 I_3} \end{cases}$$

逻辑表达式的写出并不是用德摩根定律，而是直接根据A何时得1，何时得0写出的

$$\begin{cases} A_0 = I_0 \bar{I}_1 I_2 I_3 + I_0 I_1 \bar{I}_2 \bar{I}_3 = \overline{\bar{I}_0 I_1 I_2 I_3 + I_0 I_1 \bar{I}_2 I_3} \\ A_1 = I_0 I_1 \bar{I}_2 I_3 + I_0 I_1 I_2 \bar{I}_3 = \overline{\bar{I}_0 I_1 I_2 I_3 + I_0 \bar{I}_1 I_2 I_3} \end{cases}$$



注：上图中有缓冲

3.2 BCD编码

1. 画出功能表

X_9	X_8	X_7	X_6	X_5	X_4	X_3	X_2	X_1	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	1	0	0	1

$$Y_3 = X_8 + X_9$$

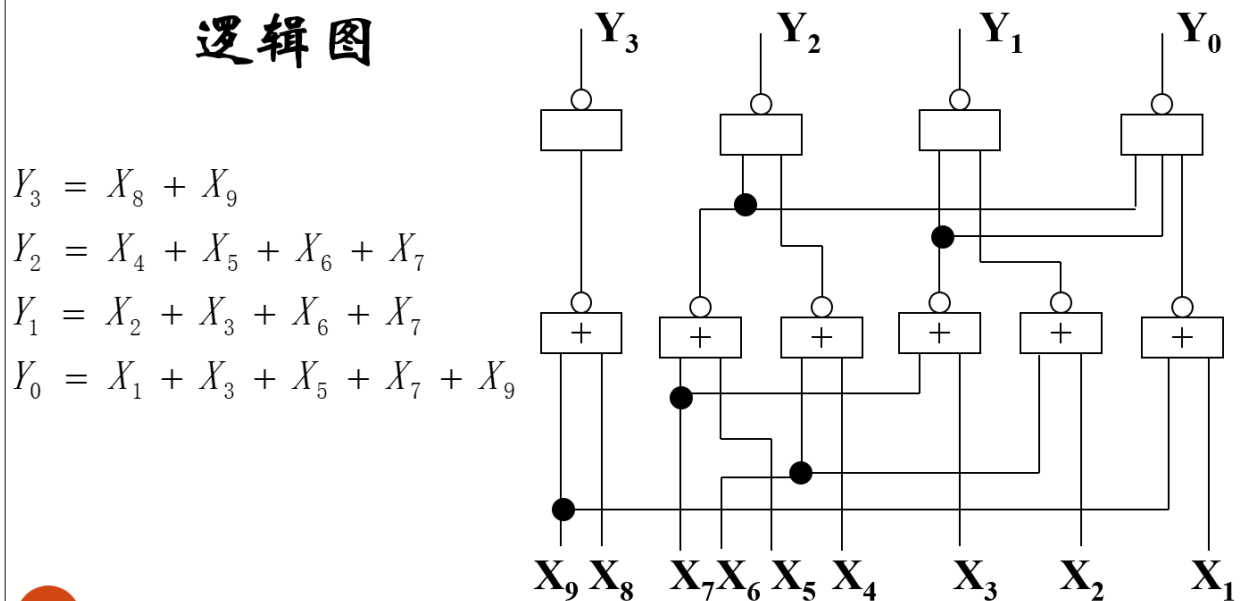
$$Y_2 = X_4 + X_5 + X_6 + X_7$$

$$Y_1 = X_2 + X_3 + X_6 + X_7$$

$$Y_0 = X_1 + X_3 + X_5 + X_7 + X_9$$

2. 画电路图

■ 根据8421码编码器输出表达式画出逻辑图



这里用到了技巧：或门=或非+非，用两个门实现了或

② 问题

上面的编码器的输入没有包含输入的全部逻辑组合，只有输入互斥时，才能用这种编码器，否则会出现这样的问题：

功能表

I_0	I_1	I_2	I_3	A_0	A_1
0	1	1	1	0	0
1	0	1	1	1	0
1	1	0	1	0	1
1	1	1	0	1	1

假设 $I_1 = "0"$, $I_2 = "0"$,

从输出表达式中可以得到
 $A_0 = "1"$ $A_1 = "1"$

产生问题的原因是：2-4编码器的功能表中没有完全包含输入的全部逻辑组合。

$$\begin{cases} A_0 = \overline{\overline{I_0}I_1I_2I_3} + \overline{I_0I_1\overline{I_2}I_3} \\ A_1 = \overline{\overline{I_0}I_1I_2I_3} + \overline{I_0\overline{I_1}I_2I_3} \end{cases}$$

I_1 和 I_2 都为0，我希望的是两个输入以我定义的某种优先级选中（中断），然而从图中可以看出，实际上输出 $A_0=A_1=1$ ，会和 $I_3=0$ 的情况相同，会选中 I_3

解决办法：优先编码器

3.3 优先编码器

以8-3优先编码器为例

✍ 优先编码器

当两条或两条以上线为“0”时，优先按输入编号大的编码，称优先编码器(Priority Encoder)

1. 8-3键盘优先编码器的设计

- 需要对各键进行编码 A_2, A_1, A_0 (8个输入的3个输出)
- 需要设定输入使能 $\bar{E}$ ，使键盘能够一个键都不按（不工作）
- 需要设定输出的编码是否有效指示 G_s
- 需要设定是否允许编码级联输出 E_o

2. 绘制功能表

$\overline{E_i}$	0	1	2	3	4	5	6	7	A_0	A_1	A_2	G_s	E_o
0	X	X	X	X	X	X	X	0	0	0	0	0	1
0	X	X	X	X	X	X	0	1	1	0	0	0	1
0	X	X	X	X	X	0	1	1	0	1	0	0	1
0	X	X	X	X	0	1	1	1	1	1	0	0	1
0	X	X	X	0	1	1	1	1	0	0	1	0	1
0	X	X	0	1	1	1	1	1	1	0	1	0	1
0	X	0	1	1	1	1	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
1	X	X	X	X	X	X	X	X	1	1	1	1	1

(A_2, A_1, A_0 用反码编码, G_s 为编码输出有效, $\overline{E_i}$ 为使能输入, E_o 为允许级联输出)

- 输出用反码 (7是000而不是111)
- 当7=0时, 由于它优先级最高, 其他按键按多少都无所谓, 为“无关项”
- 允许级联输出: 对多个键盘也有一个键盘间的优先级。如果当前高优先级键盘被按 (工作), 我希望低优先级的键盘都不工作, 所以 $E_o = 1$, 把这个输出接到其他键盘的使能端, 就可以让其他键盘都不工作。同样地, 如果高优先级键盘不工作 $E_o = 0$, 那就让低优先级键盘工作

3. 写出逻辑表达式:

$\overline{E_i}$	0	1	2	3	4	5	6	7	A_0	A_1	A_2	G_s	E_o
0	X	X	X	X	X	X	X	0	0	0	0	0	1
0	X	X	X	X	X	X	0	1	1	0	0	0	1
0	X	X	X	X	X	0	1	1	0	1	0	0	1
0	X	X	X	X	0	1	1	1	1	1	0	0	1
0	X	X	X	0	1	1	1	1	0	0	1	0	1
0	X	X	0	1	1	1	1	1	1	0	1	0	1
0	X	0	1	1	1	1	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
1	X	X	X	X	X	X	X	X	1	1	1	1	1

$$\overline{A_0} = \overline{7} + 76\overline{5} + 7654\overline{3} + 765432\overline{1} = \overline{7} + 6\overline{5} + 64\overline{3} + 642\overline{1}$$

这里的化简用到了吸收律

$\overline{E_i}$	0	1	2	3	4	5	6	7	A_0	A_1	A_2	G_s	E_0
0	X	X	X	X	X	X	X	0	0	0	0	0	1
0	X	X	X	X	X	X	0	1	1	0	0	0	1
0	X	X	X	X	X	0	1	1	0	1	0	0	1
0	X	X	X	X	0	1	1	1	1	1	0	0	1
0	X	X	X	0	1	1	1	1	0	0	1	0	1
0	X	X	0	1	1	1	1	1	1	0	1	0	1
0	X	0	1	1	1	1	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
1	X	X	X	X	X	X	X	X	1	1	1	1	1

$$\overline{E_0} = \overline{E_i} \cdot 76543210 \quad G_s = \overline{E_0} \cdot \overline{E_i}$$

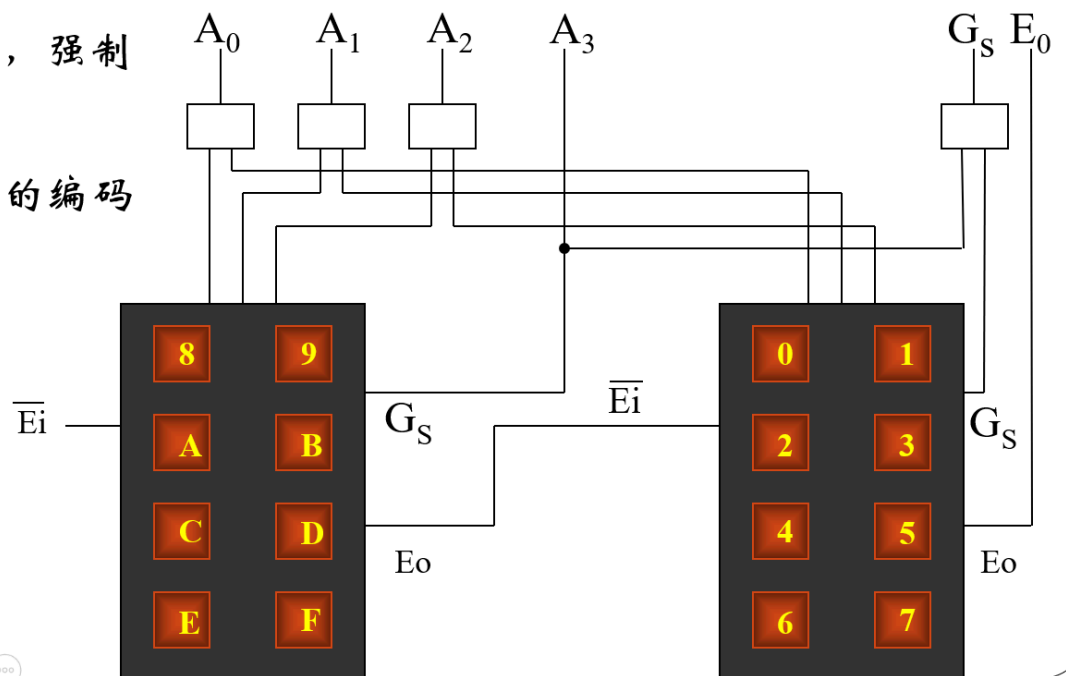
总之：

- 如果 $\overline{E_i} = 0$ ，输入有“0”，则 $G_s = "0"$ ， $\overline{E_0} = "1"$ 编码有效，级联禁止
- 如果 $\overline{E_i} = 0$ ，输入全为“1”，则 $G_s = "1"$ ， $\overline{E_0} = "0"$ 编码无效，级联有效
- 如果 $\overline{E_i} = 1$ ，无论输入为何值， $G_s = "1"$ ， $\overline{E_0} = "1"$ 编码无效，级联禁止
- 输出编码为反码，即用“000”表示7，用“111”表示0

4. 级联扩展：

高位有效，强制
低位无效

生成相应的编码
输出



需要注意的是 A_3 直接使用了高位编码器的 G_s 输出，可以验证这是刚好符合要求的原因：

- 编码使用了反码编码。高位片输出时，最高位就应该是0，低位片输出时，最高位应该是1
- $\bar{E}_i$ 的输出前面已经说过。

4 数据比较器

✎ 数据比较器

数据选择器：数字系统中能够完成数据比较功能的部件

功能：比较A、B两数，判断大于、小于、等于

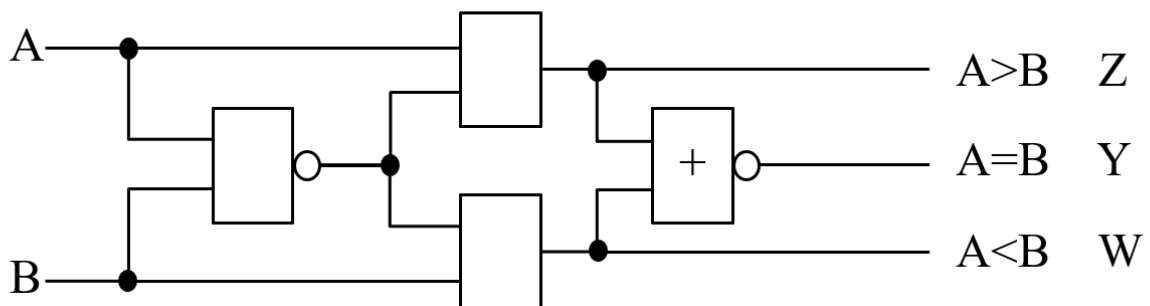
基本运算： $A_3A_2A_1A_0$ 和 $B_3B_2B_1B_0$ ，从高位开始比较，高位有不同直接分出大小。高位相等，再比较低位

4.1 一位比较器

$A_i > B_i$ 的条件： $A_i=1, B_i=0$ ；即 $A_i \cdot \bar{B}_i = 1$ 或 $Z = A_i \cdot \bar{A}_i B_i = 1$

$A_i < B_i$ 的条件： $A_i=0, B_i=1$ ；即 $\bar{A}_i \cdot B_i = 1$ 或 $W = B_i \cdot \bar{A}_i \bar{B}_i = 1$

$A_i = B_i$ 的条件： $\bar{A}_i \oplus \bar{B}_i = 1$ 或 $Y = (\bar{A}_i \cdot \bar{A}_i \cdot \bar{B}_i) + (\bar{A}_i \cdot \bar{B}_i \cdot B_i) = 1$

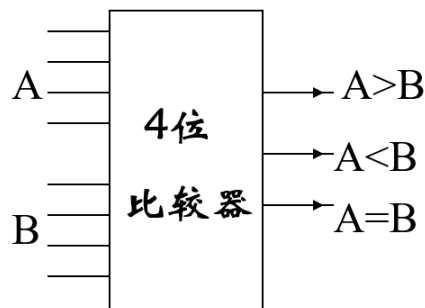


4.2 数据比较器

1. 画出功能表：

数据比较器功能表

$A_3 B_3$	$A_2 B_2$	$A_1 B_1$	$A_0 B_0$	$A > B$	$A < B$	$A = B$
$A_3 > B_3$	X	X	X	1	0	0
$A_3 < B_3$	X	X	X	0	1	0
$A_3 = B_3$	$A_2 > B_2$	X	X	1	0	0
$A_3 = B_3$	$A_2 < B_2$	X	X	0	1	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 > B_1$	X	1	0	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 < B_1$	X	0	1	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 = B_1$	$A_0 > B_0$	1	0	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 = B_1$	$A_0 < B_0$	0	1	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 = B_1$	$A_0 = B_0$	0	0	1

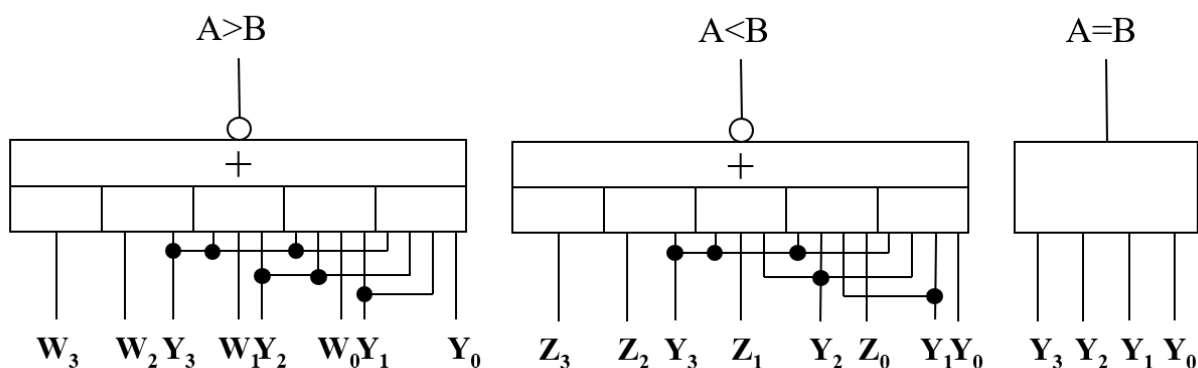


2. 做出电路图：

$$W_i = "A_i < B_i" = B_i \cdot \overline{A_i} \cdot B_i$$

$$Y_i = "A_i = B_i" = \overline{(A_i \cdot \overline{A_i} \cdot B_i) + (\overline{A_i} \cdot B_i \cdot B_i)}$$

$$Z_i = "A_i > B_i" = A_i \cdot \overline{A_i} \cdot B_i$$



5 奇偶校验器

奇偶校验器

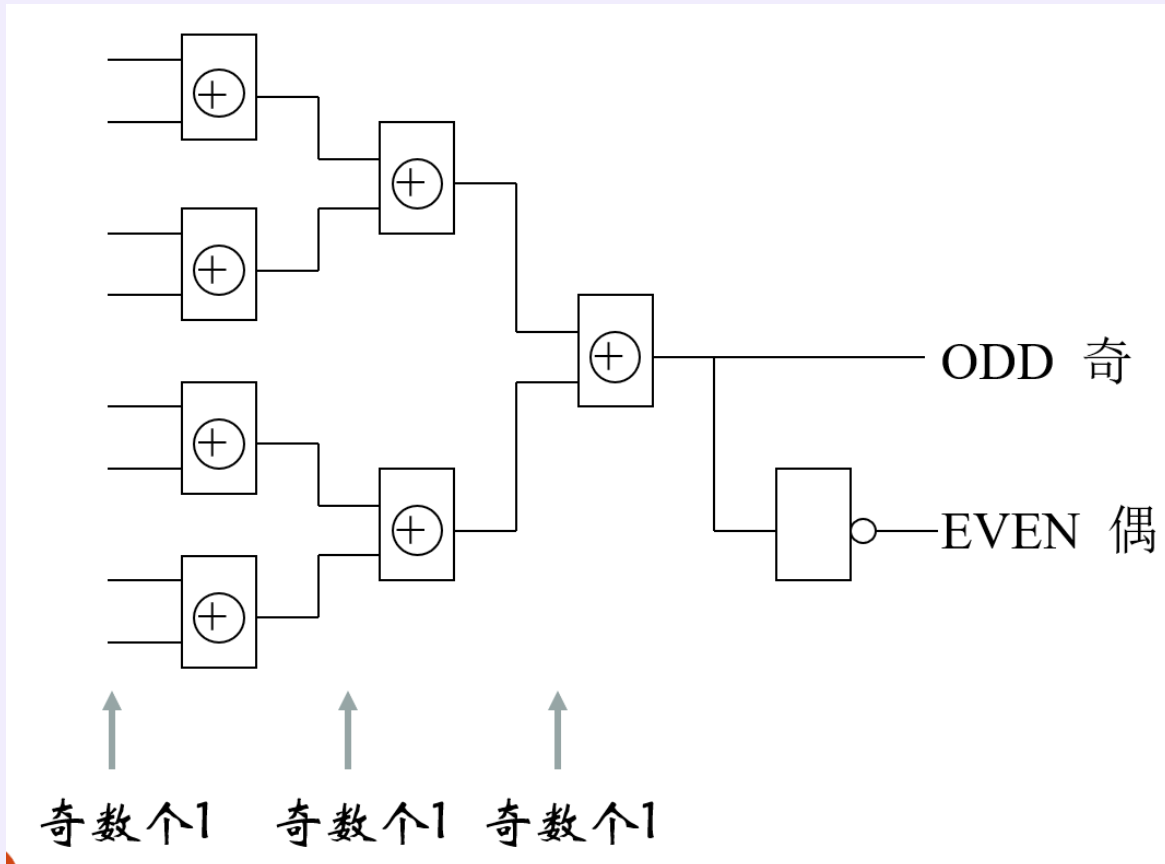
奇偶检测：检测数据中包含奇数个“1”，还是偶数个“1”

奇偶校验：发送或存储时产生奇偶校验码，在接收方或读出时进行奇偶校验，检查数据传输和记录过程中是否产生了错误

PS：奇偶校验只能发现“一位错”，且不能纠错

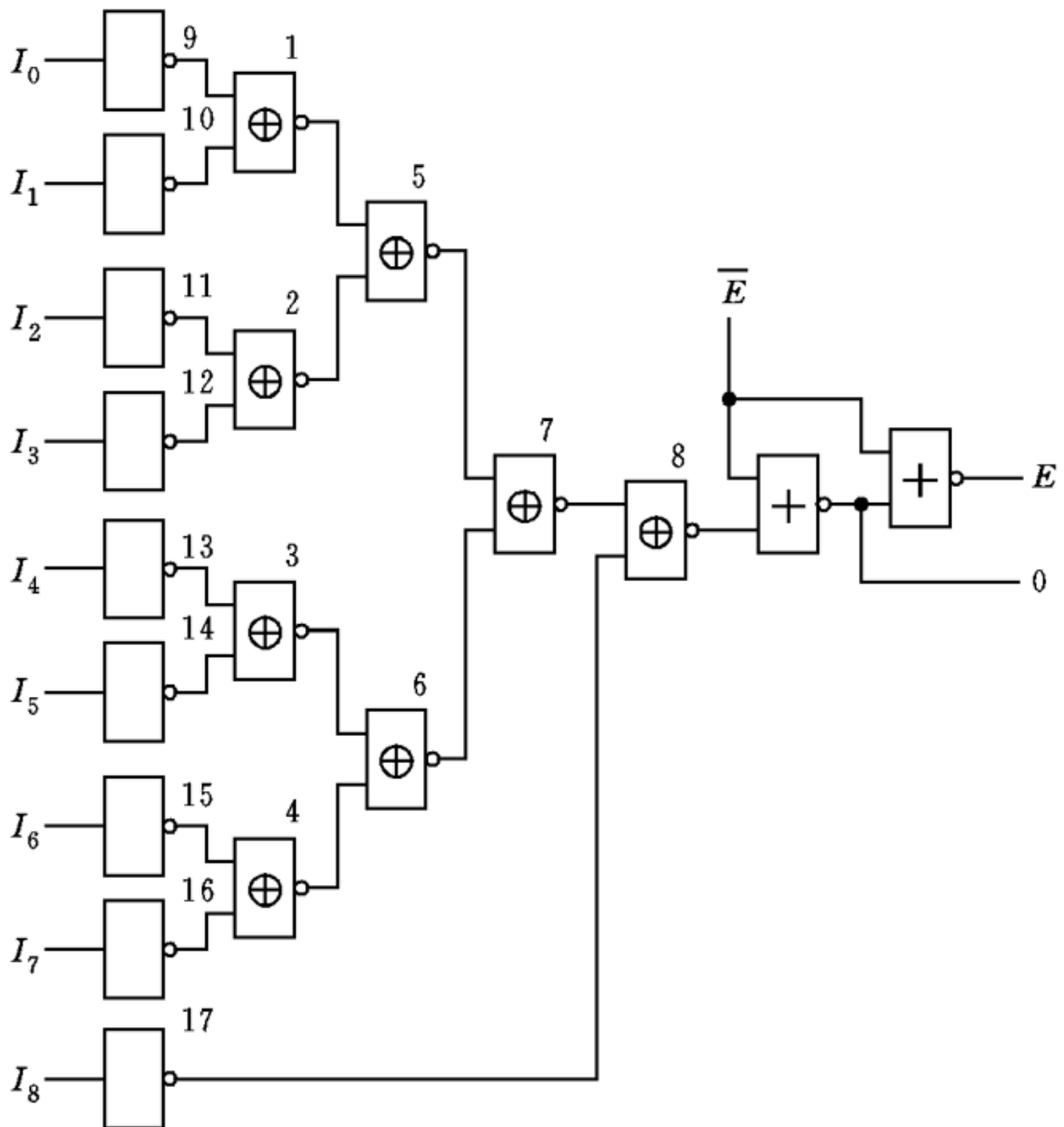
Example

1. 用异或门构成八位奇偶校验位电路



- ODD: 奇数个1时, ODD=1。EVEN=0

2. 九位奇偶校验电路



6 可编程逻辑器件

用软件设计硬件：硬件描述语言(HDL)

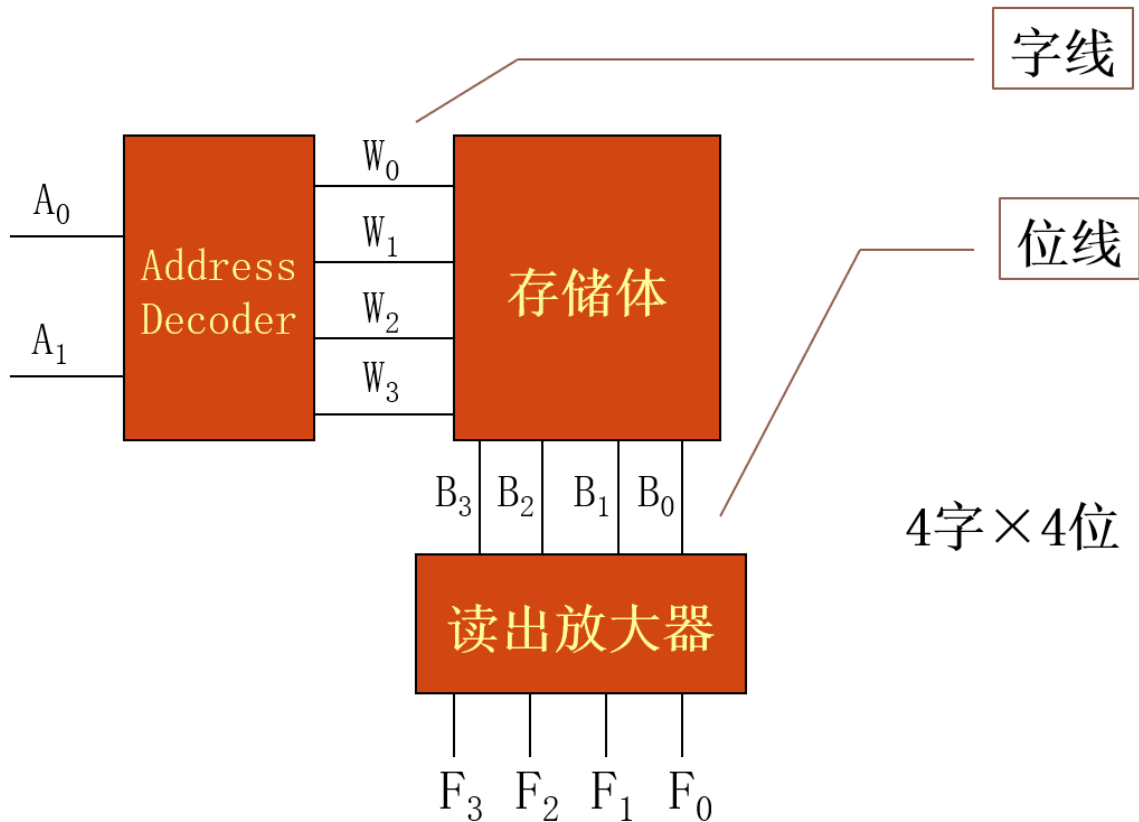
✎ 可编程逻辑器件

可编程逻辑器件PLD (Programmable Logic Device)是一大类器件的总称,包括:

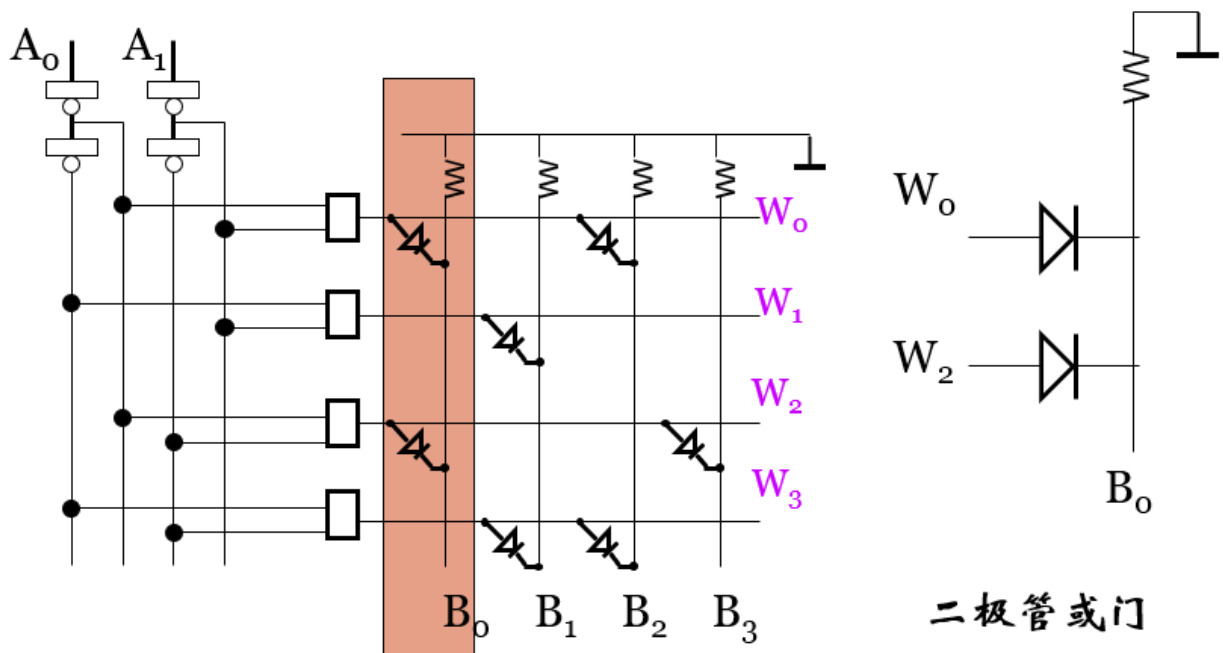
- ROM (Read-Only Memory) 只读存储器
- PLA (Programmable Logic Array) 可编程逻辑阵列
- PAL (Programmable Array Logic) 可编程阵列逻辑
- GAL (General Array Logic) 通用阵列逻辑

6.1 ROM

1. 4字×4位ROM

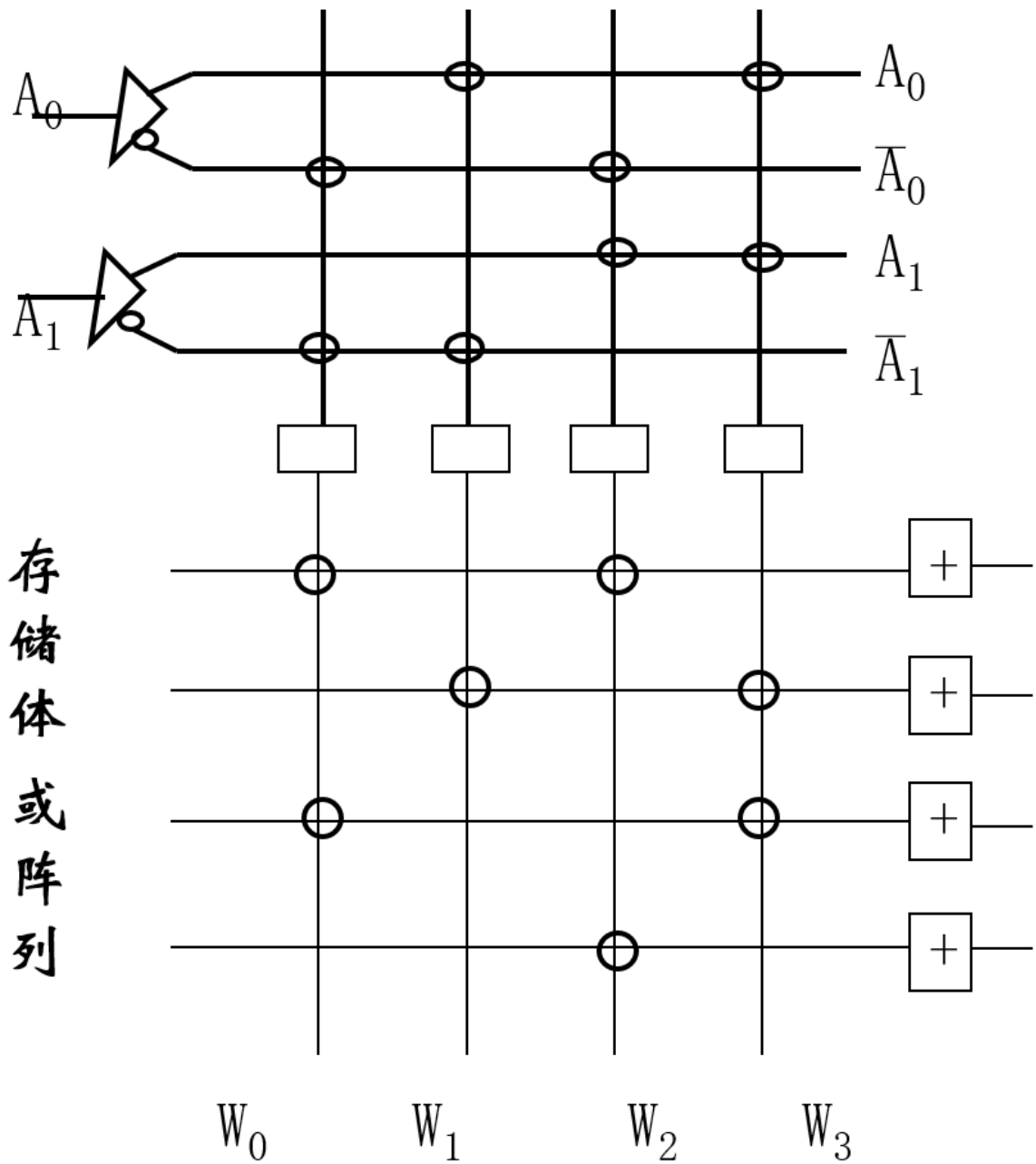


2. 实际电路图：



- 图中右边的二极管是或门的实现。比如图中，表示 $B_0 = W_0 + W_2$
- 然后，我们为了简便的绘图，把存储体用阵列作为记法。也就是把译码时的与门和存储时的或门都用方格中的 $\circ$ 表示。
- 最终得到存储逻辑结构如图：

地址译码：与阵列



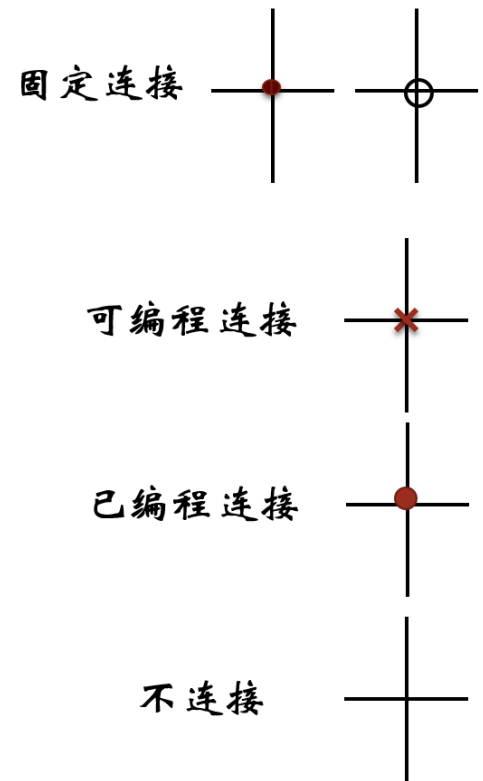
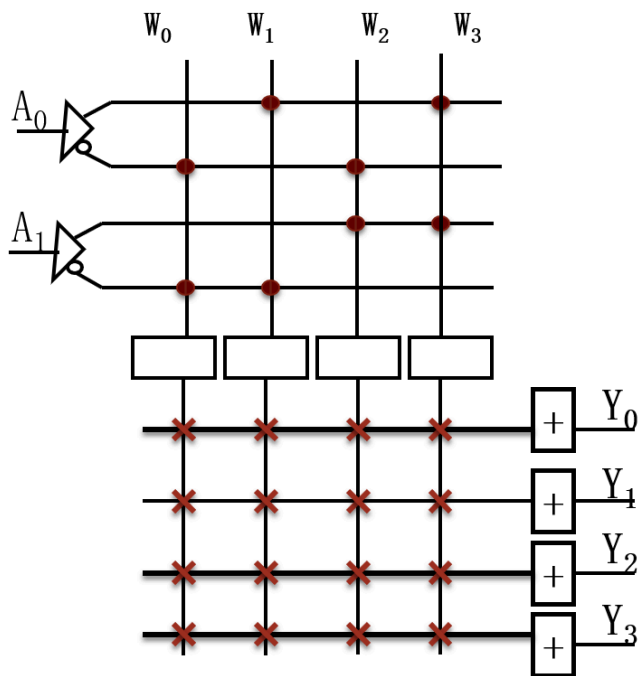
存储体或阵列

3. 实现可编程：

- 与阵列固定，或阵列可编程

- 记法：

与阵列固定、或阵列可编程



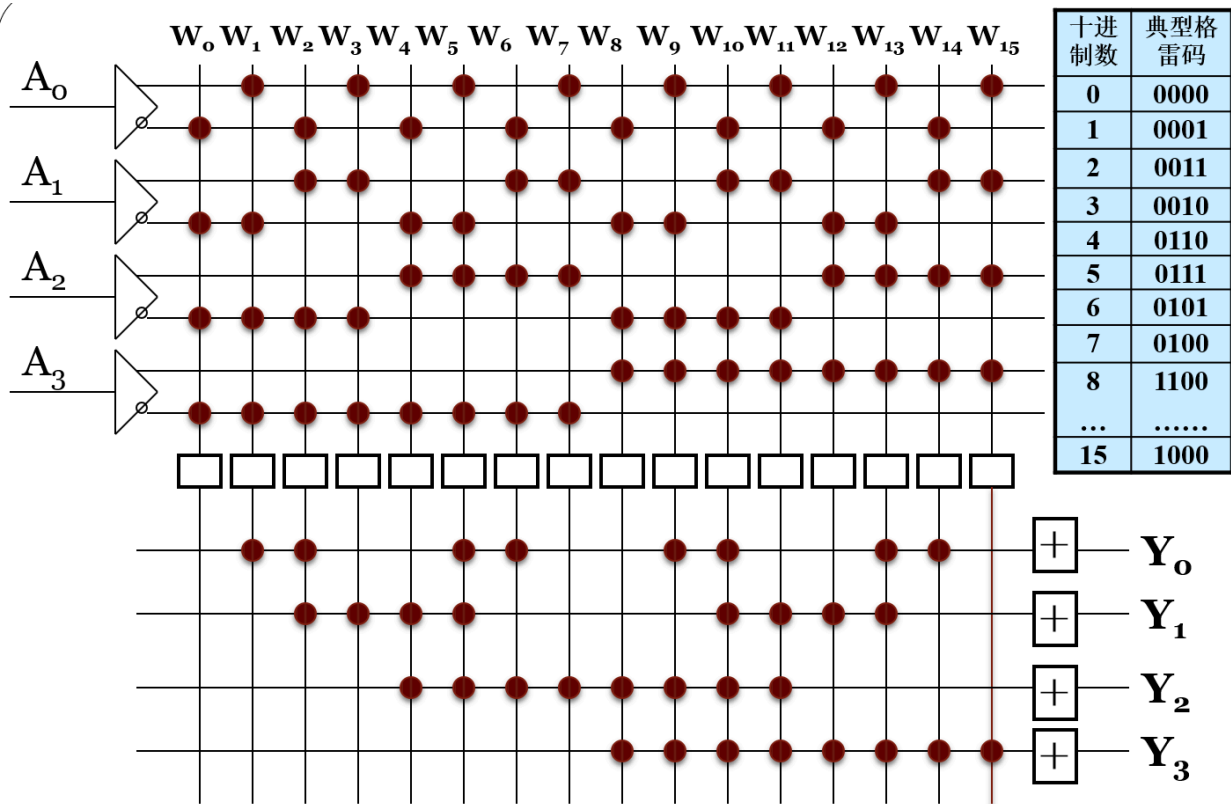
6.1.1 ROM的应用——码制转换

利用16×4ROM，将8421码→典型格雷码

- 要求：

十进制数	8421码	典型格雷码
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
.....		
15	1111	1000

2. 最终的阵列：



60

与阵列 $8 \times 16 = 128$ ，或阵列 $4 \times 16 = 64$ ，总点数192

6.2可编程逻辑阵列(PLA)

PLA

ROM的与阵列是一个译码器，是固定不可编程的

PLA的与、或阵列均可编程。即相对ROM能实现逻辑压缩，只产生需要用到的最小项。

1. 用PLA实现组合逻辑：

- 将逻辑表达式写成最简与 - 或表达式
- 用与项的“或”操作构成或阵列

三 用PLA实现码制转换 (8421->典型格雷码)

1. 8421码表示为 $B_3B_2B_1B_0$ ，格雷码表示为 $G_3G_2G_1G_0$

2. 根据前面的表格，利用卡诺图（或直接看）得到逻辑表达式：

$$G_3 = B_3$$

$$G_2 = B_3 \oplus B_2 = \overline{B_3}B_2 + B_3\overline{B_2}$$

$$G_1 = B_2 \oplus B_1 = \overline{B_2}B_1 + B_2\overline{B_1}$$

$$G_0 = B_1 \oplus B_0 = \overline{B_1}B_0 + B_1\overline{B_0}$$

3. 将所有用到的与项 P_i 提取出来，这就是与阵列需要实现的点：

$$P_0 = B_3$$

$$P_1 = \overline{B_3}B_2$$

$$P_2 = B_3\overline{B_2}$$

$$P_3 = \overline{B_2}B_1$$

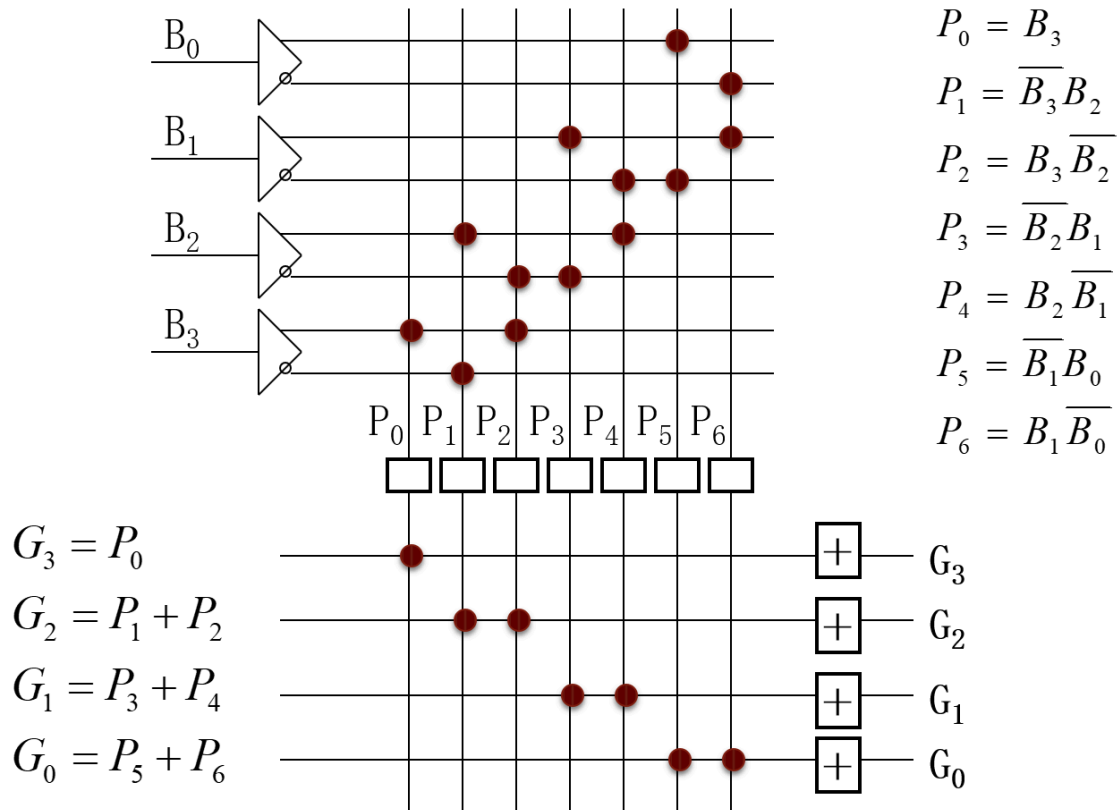
$$P_4 = B_2\overline{B_1}$$

$$P_5 = \overline{B_1}B_0$$

$$P_6 = B_1\overline{B_0}$$

4. 将 G_i 用 P_i 表示，可以得到或阵列需要表示的点

5. 得到最终的阵列图：



75

与阵列 $8 \times 7 = 56$ ，或阵列 $4 \times 7 = 28$ ，总点数 84

⚠ 注意

PLA的容量表示为：输入数 $\times$ P项数 $\times$ 输出数
这个值并不等于阵列的点数！

7 运算器（算术逻辑单元ALU）

7.1 加法器

7.1.1 一位半加器

✍ 一位半加器

不考虑低位进位输入和向高位的进位输出，两数码 X_n 、 Y_n 相加，称半加

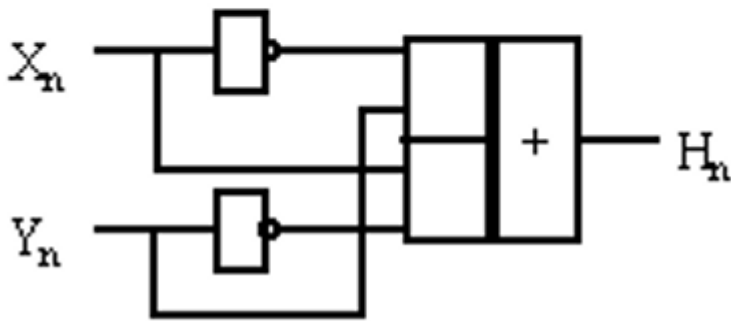
2. 功能表

X_n	Y_n	H_n
0	0	0
1	0	1
0	1	1
1	1	0

发现：一位半加器实质上就是异或门

$$H_n = \overline{X_n}Y_n + X_n\overline{Y_n} = X_n \oplus Y_n$$

$$\overline{H_n} = X_nY_n + \overline{X_n}\overline{Y_n} = \overline{X_n \oplus Y_n}$$



7.1.2 一位全加器

✎ 一位全加器

全加器（**Adder**）：将 X_n 、 Y_n 及低位进位 C_{n-1} 相加，并将进位输出 C_n ，称全加

1. 一位全加器的功能表

X_n	Y_n	C_{n-1}	F_n	C_n
0	0	0	0	0
1	0	0	1	0
0	1	0	1	0
1	1	0	0	1
0	0	1	1	0
1	0	1	0	1
0	1	1	0	1
1	1	1	1	1

2. 化简求 F_n, C_n

- F_n

$X_n Y_n$					
C_{n-1}		00	01	11	10
		0	1	0	1
0	0	0	1	0	1
1	1	1	0	1	0

$$\begin{aligned}
 F_n &= \bar{X}_n \bar{Y}_n C_{n-1} + \bar{X}_n Y_n \bar{C}_{n-1} + X_n \bar{Y}_n \bar{C}_{n-1} + X_n Y_n C_{n-1} \\
 &= \bar{C}_{n-1} (\bar{X}_n Y_n + X_n \bar{Y}_n) + C_{n-1} (\bar{X}_n \bar{Y}_n + X_n Y_n) \\
 &= \bar{C}_{n-1} (X_n \oplus Y_n) + C_{n-1} \overline{(X_n \oplus Y_n)} \\
 &= C_{n-1} \oplus (X_n \oplus Y_n)
 \end{aligned}$$

7

$X_n Y_n$					
C_{n-1}		00	01	11	10
		0	0	1	0
0	0	0	0	1	0
1	0	0	1	1	1

$$\begin{aligned}
 C_n &= X_n Y_n \bar{C}_{n-1} + X_n \bar{Y}_n C_{n-1} \\
 &\quad + \bar{X}_n Y_n C_{n-1} + X_n Y_n C_{n-1}
 \end{aligned}$$

$$\begin{aligned}
 C_n &= X_n Y_n + X_n C_{n-1} + Y_n C_{n-1} \\
 &= X_n Y_n + (X_n + Y_n) C_{n-1}
 \end{aligned}$$

$$\bar{C}_n = \bar{X}_n \bar{Y}_n + \bar{X}_n \bar{C}_{n-1} + \bar{Y}_n \bar{C}_{n-1}$$

3. 四种形式的一位全加器

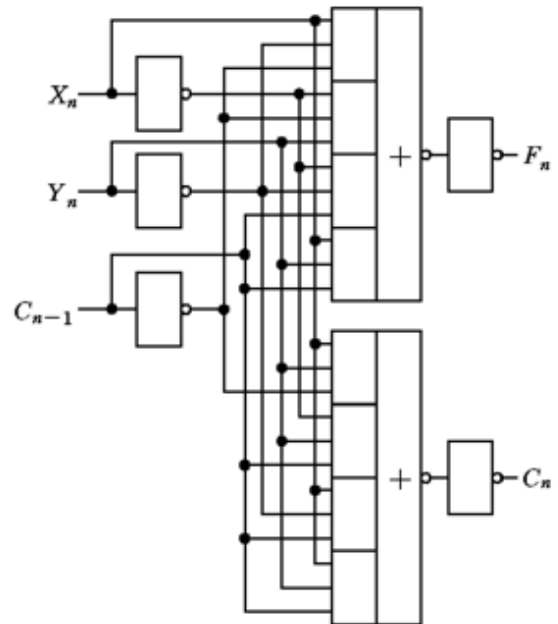
1. 不化简，全部用最小项实现，需要3级门

● 一位全加器实现方案1

$$F_n = X_n \bar{Y}_n \bar{C}_{n-1} + \bar{X}_n Y_n \bar{C}_{n-1} + \bar{X}_n \bar{Y}_n C_{n-1} + X_n Y_n C_{n-1}$$

$$C_n = X_n Y_n \bar{C}_{n-1} + X_n \bar{Y}_n C_{n-1} + \bar{X}_n Y_n C_{n-1} + X_n Y_n C_{n-1}$$

不化简，用全部最小项实现，需要3级门。

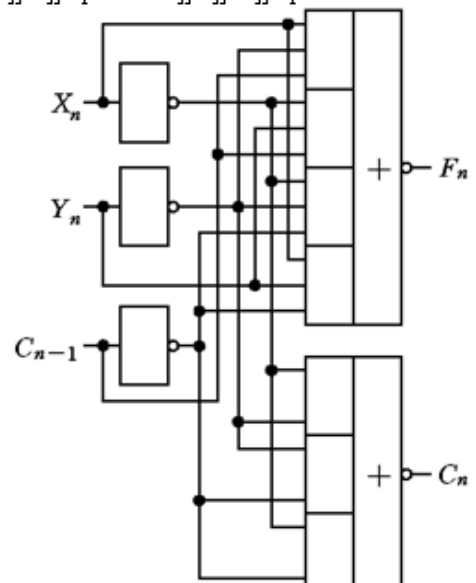


注意，倒数第二个门为与或非门

2. 化简后，只需要2级门电路，降低了延时

$$F = \bar{F} = \overline{X_n Y_n C_{n-1} + X_n Y_n C_{n-1} + X_n \bar{Y}_n C_{n-1} + X_n Y_n C_{n-1}}$$

$$C_n = \bar{C}_n = \overline{X_n Y_n + X_n C_{n-1} + Y_n C_{n-1}}$$



11

3. 4.只使用原信号，不使用反信号，使第一级非门不需要

真值表

X_n	Y_n	C_{n-1}	F_n	C_n
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

分析全加器真值表中 F_n 和 C_n 的对应关系， F_n 为“1”的条件有两个：

1、 $X_n Y_n C_{n-1}$ 均为“1”

2、 $X_n Y_n C_{n-1}$ 只有一个为“1”（有至少1个“1”且 C_n 为“0”）

用 C_n 表示 F_n ，先组合出 C_n ，再有 F_n

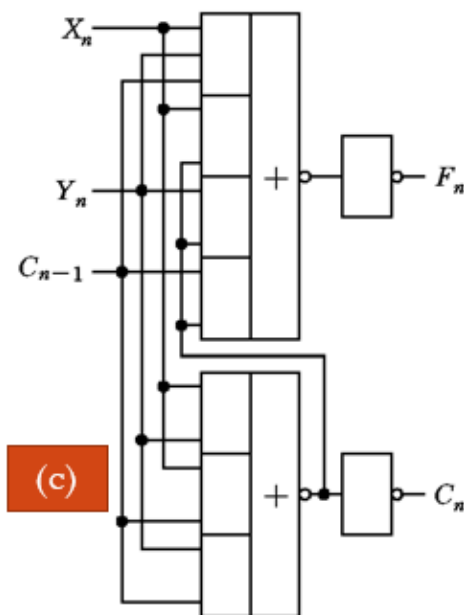
$$F_n = X_n Y_n C_{n-1} + X_n \bar{C}_n + Y_n \bar{C}_n + C_{n-1} \bar{C}_n$$

$$C_n = X_n Y_n + (X_n + Y_n) C_{n-1}$$

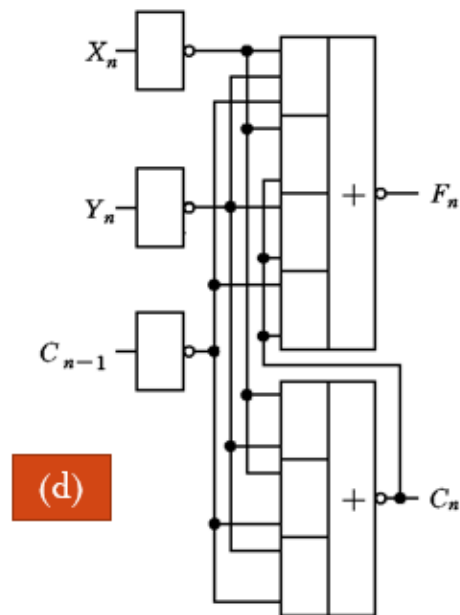
$$F_n = \bar{X}_n \bar{Y}_n \bar{C}_{n-1} + \bar{X}_n C_n + \bar{Y}_n C_n + \bar{C}_{n-1} C_n$$

$$C_n = \bar{X}_n \bar{Y}_n + (\bar{X}_n + \bar{Y}_n) \bar{C}_{n-1}$$

四种形式的全加器 (3)



C_n 的形成需要二级门延迟
 F_n 的形成需要三级门延迟



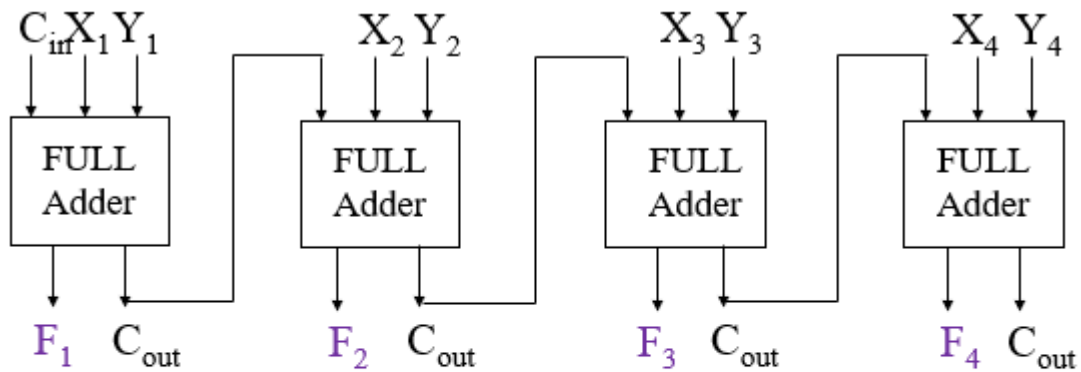
C_n 的形成需要二级门延迟
 F_n 的形成需要三级门延迟

7.1.3 四位串行进位加法器

② 思考

如何让串行进位加法器更快

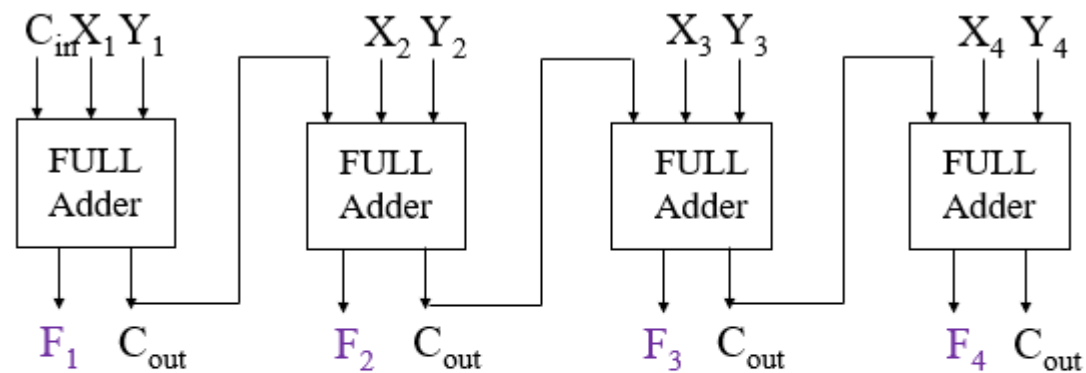
1. 使用上面的第二种一位全加器进行串行



■ 假设计算 F_n 需要2级， C_n 需要2级， C_{in} 、 X_i 和 Y_i 同时到达。最后结果需要多少级延迟？

- F_1 需要2级， C_1 需要2级， F_2 需要4级， C_2 需要4级， F_3 需要6级， C_3 需要6级， F_4 需要8级， C_4 需要8级。

2. 使用第三、第四种会更快吗？



■ 假设计算 F_n 需要3级， C_n 需要2级， C_{in} 、 X_i 和 Y_i 同时到达。最后结果需要多少级延迟？

- F_1 需要3级， C_1 需要2级， F_2 需要5级， C_2 需要4级， F_3 需要7级， C_3 需要6级， F_4 需要9级， C_4 需要8级。

事实上，观察第三、第四种全加器，发现如果二者级联，(c)的 C_n 的输入不需要经过最后一层非门，因为(d)的 C_{n-1} 输入还要再经过一层非门。这样的话可以节省两个非门

3. 使用(c)(d)级联

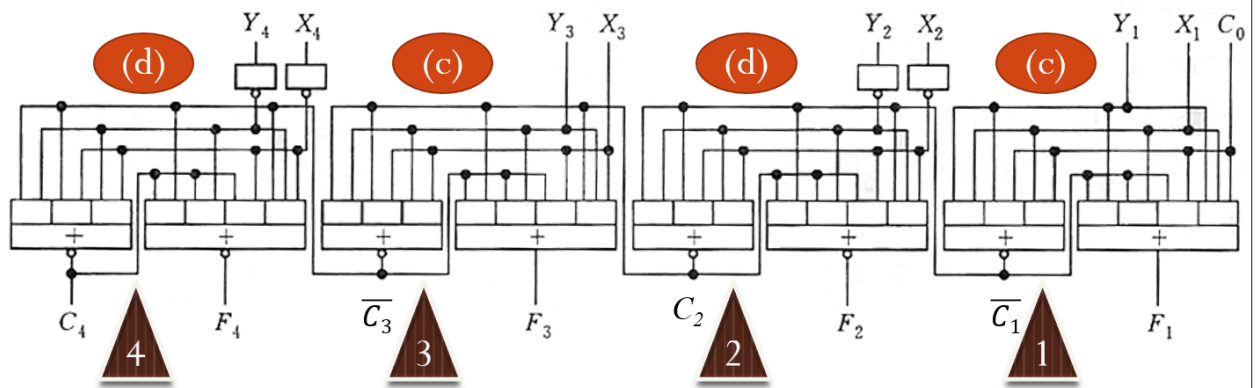


图 4-51 集成化的 4 位串行进位加法器的逻辑图

- F_1 需要 3 级， C_1 需要 2 级， F_2 需要 3 级， C_2 需要 2 级， F_3 需要 5 级， C_3 需要 4 级， F_4 需要 5 级， C_4 需要 4 级。

21

7.1.4 四位并行进位加法器

⚠ 串行的问题

前一个加法完成并提供进位后，下一个加法器才能开始算。

位数越多，需要的时间越长

理论上， C_i 只需要知道初始的 C_0 和 $X_1, Y_1, \dots, X_i, Y_i$ 就能直接算出来。考虑使用专门的电路直接把所有的进位算出来

1. 各C进位的逻辑表达式

3.3.7 运算器(21)

$G_i = X_i Y_i$ 称为

产生进位函数

四位并行加法器、超前进位加法器 $P_i = X_i + Y_i$ 称为

C_1 、 C_2 、 C_3 、 C_4 形成的条件

传递进位函数

$$C_1 = X_1 Y_1 + (X_1 + Y_1) C_0$$

$$C_2 = X_2 Y_2 + (X_2 + Y_2) X_1 Y_1 + (X_2 + Y_2) (X_1 + Y_1) C_0$$

$$C_3 = X_3 Y_3 + (X_3 + Y_3) X_2 Y_2 + (X_3 + Y_3) (X_2 + Y_2) X_1 Y_1 + (X_3 + Y_3) (X_2 + Y_2) (X_1 + Y_1) C_0$$

$$C_4 = X_4 Y_4 + (X_4 + Y_4) X_3 Y_3 + (X_4 + Y_4) (X_3 + Y_3) X_2 Y_2 + (X_4 + Y_4) (X_3 + Y_3) (X_2 + Y_2) X_1 Y_1 + (X_4 + Y_4) (X_3 + Y_3) (X_2 + Y_2) (X_1 + Y_1) C_0$$

25

- 产生进位：指这一位本身的X,Y相加产生了进位，即二者都是1
- 传递进位：指前一位产生了进位，这一位X,Y中至少有一个1，这样就会把进位传递下去

2. 进行改写：

$$C_1 = X_1 Y_1 + (X_1 + Y_1) C_0 = G_1 + P_1 C_0$$

化简，得

$$C_1 = \overline{\overline{X_1 Y_1}} + \overline{\overline{(X_1 + Y_1) C_0}}$$

改写为

$$C_1 = \overline{\overline{X_1 + Y_1 + X_1 Y_1 C_0}} = \overline{\overline{P_1 + G_1 C_0}}$$

- $P_i = X_i + Y_i$ ，代表传递进位
- $G_i = X_i Y_i$ ，代表产生进位

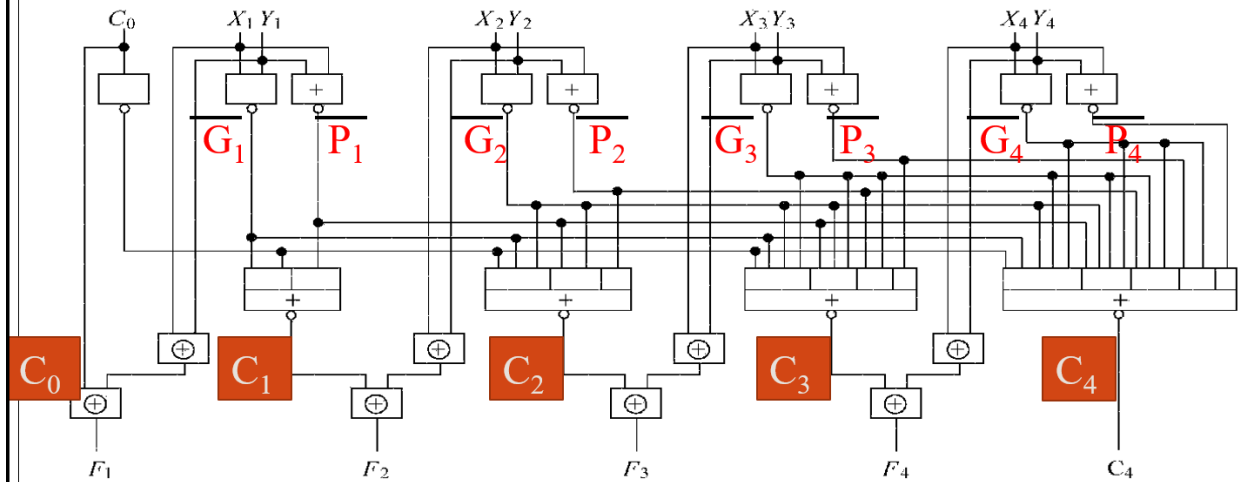
3. 逻辑电路图

$$C_1 = \overline{X_1 + Y_1 + X_1 Y_1 C_0} = \overline{P_1 + G_1 C_0}$$

$$C_2 = \overline{X_2 + Y_2 + X_2 Y_2 (X_1 + Y_1) + X_2 Y_2 X_1 Y_1 C_0} = \overline{P_2 + G_2 P_1 + G_2 G_1 C_0}$$

$$C_3 = \overline{X_3 + Y_3 + X_3 Y_3 (X_2 + Y_2) + X_3 Y_3 X_2 Y_2 (X_1 + Y_1) + X_3 Y_3 X_2 Y_2 X_1 Y_1 C_0}$$

$$C_4 = \overline{X_4 + Y_4 + X_4 Y_4 (X_3 + Y_3) + X_4 Y_4 X_3 Y_3 (X_2 + Y_2) + X_4 Y_4 X_3 Y_3 X_2 Y_2 (X_1 + Y_1) + X_4 Y_4 X_3 Y_3 X_2 Y_2 X_1 Y_1 C_0}$$



30

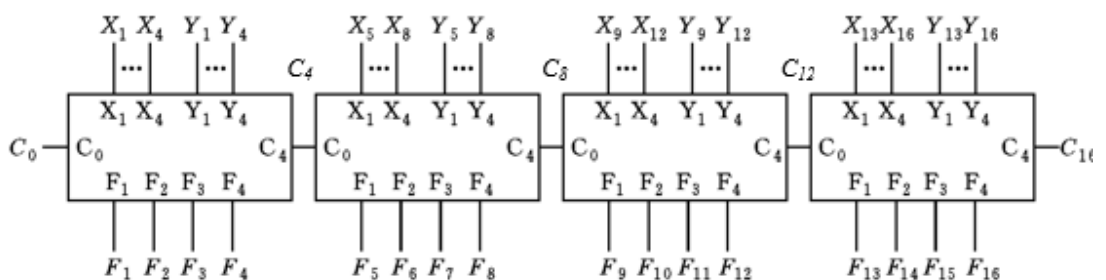
C_i 延迟级数与位数无关：都是2级； F_i 基本都是3级

7.1.5 16位加法器

1. 16位串行加法器

- 用4片四位并行加法器组成16位快速加法器。也就是说，片内是并行的，片间是串行的。
- 根据前面我们知道，四位并行加法器的C有2级门延迟，F有3级门延迟

● 用4片四位并行加法器组成16位快速加法器



此时，片内虽然是并行，但片间进位仍是串行逐片传递，产生 C_4, C_8, C_{12}, C_{16} 的延迟各是几级？ F_i 要几级？(用P136图)

C_4 需2级， $F_1 \sim F_4$ 需3级； C_8 需4级， $F_5 \sim F_8$ 需5级； C_{12} 需6级，

- $F_9 \sim F_{12}$ 需7级； C_{16} 需8级， $F_{13} \sim F_{16}$ 需9级。 $C_n = n/2$ 级
- 注意， F = 上一级的 C 的延迟 + 本身的3级

2. 16位并行进位加法器

- 类似前面把四个1位并行成4位的设计，考虑把四个4位并行成16位，对每个四位加法器进行封装。

用类似四位并行加法器中C1、C2、C3、C4形成的原理，去形成片间快速进位C4、C8、C12、C16

$$\begin{aligned} C_4 &= (G_4 + P_4G_3 + P_4P_3G_2 + P_4P_3P_2G_1) + P_4P_3P_2P_1C_0 \\ &= G_{m1} + P_{m1}C_0 \end{aligned}$$

$$\begin{aligned} C_8 &= G_8 + P_8G_7 + P_8P_7G_6 + P_8P_7P_6G_5 + P_8P_7P_6P_5C_4 \\ &= G_8 + P_8G_7 + P_8P_7G_6 + P_8P_7P_6G_5 \\ &\quad + P_8P_7P_6P_5(G_4 + P_4G_3 + P_4P_3G_2 + P_4P_3P_2G_1 + P_4P_3P_2P_1C_0) \\ &= G_{m2} + P_{m2}(G_{m1} + P_{m1}C_0) = G_{m2} + P_{m2}G_{m1} + P_{m2}P_{m1}C_0 \end{aligned}$$

- 如：C4的产生就是，第一片四位加法器产生进位 G_{m1} ，或者更低位有进位 C_0 ，这一片电路传递进位 P_{m1}

$$C_4 = G_{m1} + P_{m1}C_0$$

$$C_8 = G_{m2} + P_{m2}G_{m1} + P_{m2}P_{m1}C_0$$

$$C_{12} = G_{m3} + P_{m3}G_{m2} + P_{m3}P_{m2}G_{m1} + P_{m3}P_{m2}P_{m1}C_0$$

$$C_{16} = G_{m4} + P_{m4}G_{m3} + P_{m4}P_{m3}G_{m2} + P_{m4}P_{m3}P_{m2}G_{m1} + P_{m4}P_{m3}P_{m2}P_{m1}C_0$$

$$G_{m1} = G_4 + P_4G_3 + P_4P_3G_2 + P_4P_3P_2G_1$$

$$G_{m2} = G_8 + P_8G_7 + P_8P_7G_6 + P_8P_7P_6G_5$$

$$G_{m3} = G_{12} + P_{12}G_{11} + P_{12}P_{11}G_{10} + P_{12}P_{11}P_{10}G_9$$

$$G_{m4} = G_{16} + P_{16}G_{15} + P_{16}P_{15}G_{14} + P_{16}P_{15}P_{14}G_{13}$$

$$P_{m1} = P_4P_3P_2P_1$$

$$P_{m2} = P_8P_7P_6P_5$$

$$P_{m3} = P_{12}P_{11}P_{10}P_9$$

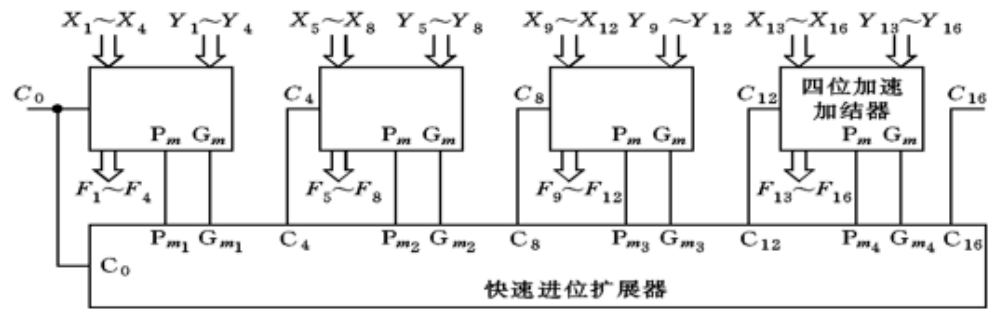
$$P_{m4} = P_{16}P_{15}P_{14}P_{13}$$

G_m 和 P_m 有规律，可以根据输入X和Y直接计算得出，意味着4位加法器内部可以直接提供 G_m 和 P_m 。C4，C8，

- 36 C12，C16可以设计一个“快速进位扩展器”形成。

- 延迟：

■ 16位快速加法器的结构图



四位并行加法器的输出提供 P_m 、 G_m 需2级延迟。产生 C_4 、 C_8 、 C_{12} 、 C_{16} 的延迟3级， $F_1 \sim F_4$ 要3级， $F_5 \sim F_{16}$ 要6级。

请大家课后比较：由1位全加器构成16位串行加法器、4位并行加法器构成16位串行加法器、16位并行加法器计算结果所需要的级数。

39

7.2 算术运算逻辑单元(ALU)

ALU

ALU是CPU的核心,不仅完成算术（加法、减法等）运算,而且完成逻辑运算（与、或、非、比较、移位）

四位ALU的核心是4位并行加法器，通过控制加法器的逻辑门或改变进位逻辑门来获得不同功能